Intro to ML - VisionGATE Solutions

GATE Data Science and Artificial Intelligence (GATE DA)

Topics Index

Click on any sub-topic below to navigate directly to it:

Document compiled for GATE DA Preparation

1 Linear Algebra Foundations

Key ideas for this section

1. **The shape rule.** A product $(m \times n)(n' \times p)$ is defined only when $n = n'$, and the result is $m \times p$. Transposition reverses both the order and the shapes: $(AB)^\top = B^\top A^\top$. Almost every "is this defined?" question is settled by writing the shapes underneath the expression and checking that adjacent inner dimensions cancel.

2. **The scalar transpose trick.** If an expression evaluates to a 1×1 quantity, it equals its own transpose. This is the single most useful identity for manipulating expressions such as $\mathbf{x}^\top A\mathbf{y}$, and it is what collapses the two cross terms when the least-squares loss is expanded.

3. **The three gradient identities.** For $\mathbf{x}, \mathbf{a} \in \mathbb{R}^n$ and $W \in \mathbb{R}^{n \times n}$:

$$\nabla_{\mathbf{x}}(\mathbf{a}^\top \mathbf{x}) = \mathbf{a}, \qquad \nabla_{\mathbf{x}}(\mathbf{x}^\top W\mathbf{x}) = (W + W^\top)\mathbf{x}, \qquad \nabla_{\mathbf{x}}(\mathbf{x}^\top \mathbf{x}) = 2\mathbf{x}$$

   The middle one simplifies to $2W\mathbf{x}$ *only* when W is symmetric.

4. **The Gram matrix.** For $X \in \mathbb{R}^{m \times n}$, the matrix $X^\top X$ is $n \times n$, symmetric and positive semi-definite, and it shares both its null space and its rank with X.

5. **Invertibility.** For a square $A \in \mathbb{R}^{n \times n}$ the following are equivalent: $\det(A) \neq 0$; the columns are linearly independent; the columns form a basis for $\mathbb{R}^n$; $\mathcal{N}(A) = \{\mathbf{0}\}$; $\operatorname{rank}(A) = n$; no eigenvalue equals 0. Note what is *not* on this list: anything about the eigenvalues being distinct.

Q1. Let $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$. Which of the following matrix operations are validly defined? (Select all that apply.)

Answer: (A, B, D)

Solution. Write the shape of every factor and check that adjacent inner dimensions match.

(A) AB — **valid.**

$$\underbrace{A}_{m \times n}\underbrace{B}_{n \times p} \longrightarrow m \times p$$

The inner n cancels.

(B) $B^\top A^\top$ — **valid.**

$$\underbrace{B^\top}_{p \times n}\underbrace{A^\top}_{n \times m} \longrightarrow p \times m$$

This is precisely $(AB)^\top$, which must be defined whenever AB is, and must have the transposed shape.

Back to 1st Page

(C) $A^\top B$ — **not valid in general.**

$$\underbrace{A^\top}_{n\times m}\ \underbrace{B}_{n\times p}$$

The inner dimensions are m and n, which need not agree. The product exists only in the special case $m = n$, so it is not *validly defined* for arbitrary A and B of the stated shapes.

(D) $ABB^\top$ — **valid.**

$$\underbrace{A}_{m\times n}\ \underbrace{B}_{n\times p}\ \underbrace{B^\top}_{p\times n} \longrightarrow m \times n$$

Both multiplications are defined, and associativity lets you read it either as $(AB)B^\top$ with shapes $(m \times p)(p \times n)$, or as $A(BB^\top)$ with shapes $(m \times n)(n \times n)$. Both give $m \times n$.

Note. *Option (C) is the useful one to remember for what it teaches: transposing a matrix does not merely relabel it, it changes which products are legal. Note also that $A^\top A$ and $BB^\top$ are* always *defined for any A and B whatsoever, since a matrix and its own transpose always have matching inner dimensions. These are the Gram matrices, and $X^\top X$ in the normal equations is exactly this construction.*

Q2. Let $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ be column vectors and $A \in \mathbb{R}^{n\times n}$ be a real matrix. Which of the following expressions is *always* identically equal to $\mathbf{x}^\top A\mathbf{y}$?

Answer: (A)

Solution. The quantity $\mathbf{x}^\top A\mathbf{y}$ has shape $(1 \times n)(n \times n)(n \times 1) = 1 \times 1$. It is a scalar, and a scalar equals its own transpose. Therefore

$$\mathbf{x}^\top A\mathbf{y} = \left(\mathbf{x}^\top A\mathbf{y}\right)^\top = \mathbf{y}^\top A^\top \mathbf{x}$$

using $(PQR)^\top = R^\top Q^\top P^\top$ and $(\mathbf{x}^\top)^\top = \mathbf{x}$. This is option (A), and it holds for every A, with no assumptions.

Why (B) and (C) fail. Applying the same trick to option (B) gives $\mathbf{y}^\top A\mathbf{x} = \mathbf{x}^\top A^\top \mathbf{y}$, which is option (C). So (B) and (C) are equal to each other, and both equal $\mathbf{x}^\top A\mathbf{y}$ only when $A^\top = A$. Symmetry was not assumed.

A concrete counterexample in $\mathbb{R}^2$. Take

$$A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Then $\mathbf{x}^\top A\mathbf{y} = A_{12} = 1$, whereas $\mathbf{y}^\top A\mathbf{x} = A_{21} = 0$ and $\mathbf{x}^\top A^\top \mathbf{y} = A_{21} = 0$. Since $1 \neq 0$, options (B) and (C) are both false, and option (D) — which asserts (B) as well as (A) — falls with them.

Back to 1st Page

Note. *Selecting $\mathbf{x}^\top A\mathbf{y}$ from a matrix is worth seeing explicitly: $\mathbf{e}_i^\top A\mathbf{e}_j = A_{ij}$, so the expression picks out row i, column j. Read that way, option (A) says $A_{ij} = (A^\top)_{ji}$, which is the definition of the transpose, while option (B) would say $A_{ij} = A_{ji}$, which is the definition of* symmetry *— an extra assumption, not an identity. This same scalar-transpose argument is what merges the two cross terms $-\mathbf{y}^\top X\mathbf{w}$ and $-\mathbf{w}^\top X^\top\mathbf{y}$ when the least-squares loss $(\mathbf{y}-X\mathbf{w})^\top(\mathbf{y}-X\mathbf{w})$ is expanded.*

Q3. Let $\mathbf{x}, \mathbf{a} \in \mathbb{R}^n$ be column vectors and $W \in \mathbb{R}^{n\times n}$ be a symmetric matrix. Which of the following gradient identities with respect to $\mathbf{x}$ are correct? (Select all that apply.)

Answer: (A, C, D)

Solution. **(A)** $\nabla_{\mathbf{x}}(\mathbf{a}^\top\mathbf{x}) = \mathbf{a}$ — **correct.** Write the expression componentwise as $\mathbf{a}^\top\mathbf{x} = \sum_{i=1}^n a_i x_i$. Differentiating with respect to x_k leaves only the k-th term, giving a_k. Collecting the components gives $\mathbf{a}$. A linear function has a constant gradient, exactly as $\frac{d}{dx}(ax) = a$ in one dimension.

(C) $\nabla_{\mathbf{x}}(\mathbf{x}^\top W\mathbf{x}) = 2W\mathbf{x}$ — **correct.** In general, for any square W,

$$\nabla_{\mathbf{x}}(\mathbf{x}^\top W\mathbf{x}) = (W + W^\top)\mathbf{x}$$

because $\mathbf{x}$ appears twice and the product rule contributes one term for each occurrence. Since W is given as symmetric, $W^\top = W$ and the two terms coincide:

$$(W + W^\top)\mathbf{x} = 2W\mathbf{x}$$

(B) $\nabla_{\mathbf{x}}(\mathbf{x}^\top W\mathbf{x}) = W\mathbf{x}$ — **incorrect.** This drops the factor of 2, accounting for only one of the two occurrences of $\mathbf{x}$. The one-dimensional analogue makes the error plain: with $n = 1$ the expression is wx^2, whose derivative is $2wx$, not wx. Note that (B) and (C) cannot both be right, since they differ by a factor of 2 for every $\mathbf{x} \neq \mathbf{0}$.

(D) $\nabla_{\mathbf{x}}(\mathbf{x}^\top\mathbf{x}) = 2\mathbf{x}$ — **correct.** This is the special case $W = I$ of identity (C). The identity matrix is symmetric, so the result is $2I\mathbf{x} = 2\mathbf{x}$. Componentwise, $\mathbf{x}^\top\mathbf{x} = \sum_i x_i^2$ and $\partial/\partial x_k = 2x_k$.

Note. *These three identities are the entire toolkit needed to derive the normal equations. Expanding gives $J(\mathbf{w}) = \mathbf{y}^\top\mathbf{y} - 2\mathbf{y}^\top X\mathbf{w} + \mathbf{w}^\top(X^\top X)\mathbf{w}$; identity (A) handles the linear term and identity (C) the quadratic one, using the fact that $X^\top X$ is symmetric. Setting the result to zero yields $X^\top X\mathbf{w} = X^\top\mathbf{y}$. The symmetry hypothesis on W is doing real work here, and it is satisfied precisely because the matrix arising in least squares is a Gram matrix.*

Q4. Let $X \in \mathbb{R}^{m\times n}$. Which of the following statements is FALSE?

Back to 1st Page

Answer: (D)

Solution. The cleanest route through this question is to prove statement (B) first. Once the two null spaces are known to be identical, statements (A) and (C) both follow with almost no extra work, and (D) can then be dismissed on dimensional grounds.

(B) $\mathcal{N}(X) = \mathcal{N}(X^\top X)$ — true. The null space of a matrix A is the set of all vectors $\mathbf{v}$ with $A\mathbf{v} = \mathbf{0}$. Proving two sets identical requires showing each is contained in the other.

Part 1: $\mathcal{N}(X) \subseteq \mathcal{N}(X^\top X)$. Suppose $\mathbf{v} \in \mathcal{N}(X)$, so that $X\mathbf{v} = \mathbf{0}$. Multiplying on the left by $X^\top$,

$$X^\top(X\mathbf{v}) = X^\top\mathbf{0} \implies X^\top X\mathbf{v} = \mathbf{0}$$

Hence $\mathbf{v} \in \mathcal{N}(X^\top X)$.

Part 2: $\mathcal{N}(X^\top X) \subseteq \mathcal{N}(X)$. Suppose instead that $X^\top X\mathbf{v} = \mathbf{0}$. Multiplying on the left by $\mathbf{v}^\top$ and regrouping by associativity,

$$\mathbf{v}^\top\left(X^\top X\mathbf{v}\right) = 0 \implies (X\mathbf{v})^\top(X\mathbf{v}) = 0 \implies \|X\mathbf{v}\|_2^2 = 0$$

A vector's squared magnitude vanishes only when the vector itself is zero, so $X\mathbf{v} = \mathbf{0}$ and $\mathbf{v} \in \mathcal{N}(X)$.

Both inclusions hold, so the two null spaces are identical.

(A) $\operatorname{rank}(X^\top X) = \operatorname{rank}(X)$ — true. This follows from (B) together with the rank–nullity theorem, which states that for any matrix A with n columns,

$$\operatorname{rank}(A) + \operatorname{nullity}(A) = n$$

The matrix X is $m \times n$ and so has n columns, while $X^\top X$ is $(n \times m)(m \times n) = n \times n$ and therefore also has n columns. Applying the theorem to each,

$$\operatorname{rank}(X) + \operatorname{nullity}(X) = n, \qquad \operatorname{rank}(X^\top X) + \operatorname{nullity}(X^\top X) = n$$

By (B) the two null spaces are the same set, so they have the same dimension: $\operatorname{nullity}(X) = \operatorname{nullity}(X^\top X)$. Substituting gives

$$n - \operatorname{rank}(X) = n - \operatorname{rank}(X^\top X) \implies \operatorname{rank}(X) = \operatorname{rank}(X^\top X)$$

(C) Linearly independent columns $\Rightarrow$ $X^\top X$ invertible — true. If the columns of X are linearly independent, no column is a linear combination of the others, which is precisely the statement that $X\mathbf{v} = \mathbf{0}$ has only the trivial solution. Hence $\mathcal{N}(X) = \{\mathbf{0}\}$, and by (B) also $\mathcal{N}(X^\top X) = \{\mathbf{0}\}$.

Back to 1st Page

Since $X^\top X$ is square, having a trivial null space is equivalent to invertibility, so $X^\top X$ is invertible.

(D) $\text{rank}(X^\top X)$ **is always equal to** m **— FALSE.** This violates the dimensional limits on rank. The product $X^\top X$ is an $n \times n$ matrix, and the rank of any matrix cannot exceed its smaller dimension, so

$$\text{rank}(X^\top X) \leq n$$

Combined with (A), the correct statement is $\text{rank}(X^\top X) = \text{rank}(X) \leq \min(m, n)$, which can equal m only in the special case $m \leq n$.

Consider a typical machine learning dataset with $m = 1000$ samples and $n = 5$ features. Then $X^\top X$ is a 5×5 matrix whose rank is at most 5, so it is impossible for its rank to equal $m = 1000$. Since $m > n$ is the usual situation in regression, the claim fails on shape grounds alone in exactly the case that matters most.

Note. *Statement (C) is exactly the existence condition for the closed-form solution* $\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$*, which is why this chain of results underpins ordinary least squares. It also explains why the closed form fails when* $n > m$ *— more features than samples — since* $\text{rank}(X) \leq m < n$ *and* $X^\top X$ *is then singular no matter how the data happens to be arranged. Ridge regression sidesteps this by solving* $(X^\top X + \lambda I)\mathbf{w} = X^\top \mathbf{y}$*, which is invertible for every* $\lambda > 0$ *because* $X^\top X$ *is positive semi-definite and adding* λI *shifts all its eigenvalues up by* λ*.*

Q5. Consider a square matrix $A \in \mathbb{R}^{n \times n}$. Which of the following conditions independently guarantee that A is invertible? (Select all that apply.)

Answer: (A, B, D)

Solution. **(A)** $\det(A) \neq 0$ **— guarantees invertibility.** This is the classical criterion; the adjugate formula $A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$ is well defined precisely when the determinant is non-zero.

(B) The columns form a basis for $\mathbb{R}^n$ **— guarantees invertibility.** A basis is by definition linearly independent and spanning, so the n columns are independent, $\text{rank}(A) = n$, and A is invertible. Equivalently, A maps $\mathbb{R}^n$ onto $\mathbb{R}^n$ bijectively.

(D) $\mathcal{N}(A) = \{\mathbf{0}\}$ **— guarantees invertibility.** A trivial null space makes the map injective, and by rank–nullity $\text{rank}(A) = n - 0 = n$. For a *square* matrix injectivity forces surjectivity, so A is invertible.

(C) A **has exactly** n **distinct real eigenvalues — does NOT guarantee invertibility.** Distinct eigenvalues guarantee a different property: n distinct eigenvalues give n linearly independent eigenvectors, so A is

Back to 1st Page

diagonalisable. That says nothing about whether any eigenvalue is zero, and invertibility is governed by

$$\det(A) = \prod_{i=1}^{n} \lambda_i$$

which vanishes as soon as a single $\lambda_i = 0$. A counterexample in $\mathbb{R}^{2\times 2}$:

$$A = \begin{bmatrix} 0 & 0 \\ 0 & 1 \end{bmatrix}$$

Its eigenvalues are 0 and 1: two distinct real values, as required. But $\det(A) = 0$, the first column is the zero vector, and A is singular.

Note. *The correct eigenvalue criterion is "A has no zero eigenvalue", which is neither implied by nor implies distinctness. The two properties are genuinely independent: the identity matrix I is invertible with all eigenvalues repeated, while* $\mathrm{diag}(0, 1)$ *has distinct eigenvalues and is singular. Options (A), (B) and (D) are three of the many equivalent statements in the invertible matrix theorem — which is why each one* independently *suffices, exactly as the question's wording invites you to check.*

Back to 1st Page

2 Model Formulation

Key ideas for this section

1. **"Linear" means linear in the parameters.** A model is a linear model when the prediction is a linear combination of the weights, regardless of how the *features* are constructed. Squares, logs, sines and products of features are all fair game, because they are computed before the weights ever enter.

2. **The bias trick.** Appending a column of ones to X and a bias entry to $\mathbf{w}$ turns $\mathbf{y} = X\mathbf{w} + b$ into $\mathbf{y} = X_{\text{aug}}\mathbf{w}_{\text{aug}}$. Omitting it forces the fitted hyperplane through the origin.

3. **Coefficients are partial effects.** β_j is the change in $\hat{y}$ per one-unit rise in x_j *with every other feature in the model held fixed.* It is not a correlation, not a variance share, and not an importance score.

4. **Coefficients carry units.** Rescaling a feature rescales its coefficient inversely, leaving the fitted values untouched. OLS predictions are scale-invariant; OLS coefficients emphatically are not.

5. **Invertibility of $X^\top X$.** Any linear dependence among the columns of the design matrix — including a constant column duplicating the appended ones column — makes $X^\top X$ singular.

Q1. In order to concisely represent a linear regression model with a bias term (i.e., $y = Xw + b$) as a single matrix-vector multiplication $y = X_{\text{aug}}w_{\text{aug}}$, what transformations must be applied to the design matrix $X \in \mathbb{R}^{N\times D}$ and the weight vector $w \in \mathbb{R}^D$? (Select all that apply)

Answer: (A, C)

Solution. This is the standard *bias trick*. For a single data point the model reads

$$y_i = w_1x_{i1} + w_2x_{i2} + \cdots + w_Dx_{iD} + b$$

and the aim is to fold the trailing $+b$ into the matrix product itself, so that no separate additive term has to be carried through the algebra. This requires one change to the inputs and one matching change to the weights.

(A) A column of ones is appended to X — correct. The design matrix has shape $N \times D$, so appending a constant column gives

$$X_{\text{aug}} \in \mathbb{R}^{N\times(D+1)}$$

In the product $X_{\text{aug}}\mathbf{w}_{\text{aug}}$ this column is multiplied by the bias entry of the weight vector, contributing $1 \cdot b$ to every row at once. The bias is thereby added to every prediction as a consequence of the multiplication rather than as a separate step. The column may be placed first or last; only the position of b within $\mathbf{w}_{\text{aug}}$ changes accordingly.

Back to 1st Page

(C) The weight vector is expanded by one dimension — correct. The extra column in X_{aug} must be matched by an extra entry in the weights:

$$\mathbf{w}_{\text{aug}} = \begin{bmatrix} w_1 & w_2 & \cdots & w_D & b \end{bmatrix}^\top \in \mathbb{R}^{(D+1)\times 1}$$

The shapes now agree, since $\big(N \times (D+1)\big)\big((D+1) \times 1\big) = N \times 1$, giving N predictions with b correctly included in each.

(B) A row of ones is appended to X — incorrect. This changes the shape to $(N+1) \times D$. In a design matrix a *row* is one sample, so this injects a fictitious observation whose every feature equals 1 — it corrupts the dataset rather than the model. It also breaks the algebra outright: an $(N+1) \times D$ matrix cannot multiply a $(D+1) \times 1$ weight vector, since the inner dimensions D and $D+1$ do not match.

(D) The target vector y is augmented — incorrect. The target holds the ground-truth values being predicted. Augmenting it would make it $N \times 2$, changing the shape of the labels themselves. The bias belongs to the model, not to the data being predicted, so the fix applies to the inputs and weights and never to the targets.

Options (A) and (C) are a matched pair and must be applied together; either alone leaves the dimensions mismatched.

Note. *Once the ones column is present, one gradient formula covers every weight including the bias, since b is now simply the weight on a feature that always equals 1 — the case $p = 0$ with $x_0 = 1$. This uniformity is the real payoff of the trick, and it is why almost every derivation in this course assumes the augmented form from the outset.*

Q2. In order to concisely represent a linear regression model with a bias term (i.e., $y = Xw + b$) as a single matrix-vector multiplication $y = X_{\text{aug}} w_{\text{aug}}$, what transformations must be applied to the design matrix $X \in \mathbb{R}^{N \times D}$ and the weight vector $w \in \mathbb{R}^D$?

Answer: (A, C)

Solution. This is the standard *bias trick*. For a single data point $\mathbf{x}_i$, the linear model with a bias term is

$$y_i = w_1 x_{i1} + w_2 x_{i2} + \cdots + w_D x_{iD} + b$$

and the aim is to fold the trailing $+b$ into the matrix product itself, so that no separate additive term has to be carried through the algebra. Doing so requires one change to the inputs and one matching change to the weights.

(A) A column of ones is appended to the design matrix X — correct. The original design matrix has shape $N \times D$, where N is the number of samples and D the number of features. Appending a column

Back to 1st Page

of 1s — on either side, though the left is the more common convention — produces

$$X_{\text{aug}} \in \mathbb{R}^{N\times(D+1)}$$

In the product $X_{\text{aug}}\mathbf{w}_{\text{aug}}$, this constant column is multiplied by the bias entry of the augmented weight vector, contributing $1 \cdot b$ to every row at once. The bias is thereby added to every prediction simultaneously, as a consequence of the multiplication rather than as a separate step.

(C) The weight vector w is expanded by one dimension to include the bias scalar b — correct. The original weight vector is $D \times 1$, one weight per feature. The extra column in X_{aug} must be matched by an extra entry in the weights:

$$\mathbf{w}_{\text{aug}} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_D \\ b \end{bmatrix} \in \mathbb{R}^{(D+1)\times 1}$$

The shapes now agree, since $(N \times (D+1))\big((D+1) \times 1\big) = N \times 1$, giving a vector of N predictions with b correctly included in each.

(B) A row of ones is appended to the design matrix X — incorrect. This would change the shape to $(N+1) \times D$. In a design matrix a *row* is one sample, so this amounts to injecting a fictitious observation whose every feature equals 1 — it corrupts the dataset rather than the model. It also breaks the algebra outright: an $(N+1) \times D$ matrix cannot multiply a $(D+1) \times 1$ weight vector, since the inner dimensions D and $D+1$ do not match.

(D) The target vector y is augmented with a column of ones — incorrect. The target vector, of shape $N \times 1$, holds the ground-truth values being predicted. Augmenting it would make it $N \times 2$, changing the shape of the labels themselves. The bias belongs to the model, not to the data being predicted, so the fix must be applied to the inputs and weights — never to the targets.

Summary. Options (A) and (C) are a matched pair and must be applied together: a column of 1s is added to the inputs, and the bias b is appended to the weights. Either one alone leaves the dimensions mismatched.

Note. *Two consequences of the ones column are worth knowing. First, it is what forces the least-squares residuals to sum to zero: the normal equations give $X^\top(\mathbf{y} - X\mathbf{w}) = \mathbf{0}$, and applying this to the constant column yields $\sum_i r_i = 0$. Drop the intercept and that guarantee disappears. Second, when the features are scaled the ones column must be left alone, and*

Back to 1st Page

VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

when a penalty is applied the bias entry is normally excluded from it — penalising b would tie the fitted model to where the origin happens to sit, which is an arbitrary choice of units rather than a property of the data.

Q3. Which of the following statements regarding polynomial regression are true? (Select all that apply)

Answer: (A, C)

Solution. (A) — true. The word "linear" in linear regression refers to linearity in the *parameters*, not in the inputs. Writing

$$\hat{y} = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M$$

the prediction is a linear combination of $w_0, \ldots, w_M$ with coefficients $1, x, x^2, \ldots, x^M$. Those coefficients are computed from the data before any weight enters, so as far as the optimisation is concerned they are just numbers filling the columns of a design matrix.

(C) — true. This is the entire purpose of the construction. Although the model is linear in $\mathbf{w}$, the fitted function of x is a curve, so genuinely non-linear relationships in the original feature space can be captured. A straight line in the transformed space $(1, x, x^2, \ldots)$ corresponds to a polynomial curve in the original space.

(B) — false. The squared-error loss remains

$$J(\mathbf{w}) = \|\mathbf{y} - \Phi\mathbf{w}\|_2^2$$

where Φ is the design matrix of polynomial features. This is a convex quadratic in $\mathbf{w}$ with curvature matrix $\Phi^\top\Phi$, which is positive semi-definite regardless of what the columns of Φ contain. Convexity is a property of how the weights enter, and transforming the features does not disturb it. There is still exactly one global minimum and no local minima to be trapped in.

(D) — false. Since the problem is ordinary least squares on the matrix Φ, the closed form applies verbatim:

$$\mathbf{w} = (\Phi^\top\Phi)^{-1}\Phi^\top\mathbf{y}$$

Every result derived for OLS — the normal equations, residual orthogonality, the projection interpretation — carries over with X replaced by Φ.

Note. *The practical difficulty with high-degree polynomials is not solvability but conditioning and overfitting. The columns $1, x, x^2, \ldots$ are strongly correlated over a narrow range of x, so $\Phi^\top\Phi$ becomes badly conditioned as*

Back to 1st Page

the degree grows; orthogonal polynomial bases such as Legendre or Chebyshev are used to mitigate this. Separately, a degree-9 polynomial fitted to 11 points will pass very close to every one of them while oscillating wildly between them — the standard illustration of high variance.

Q4. Which of the following mathematical formulations can be solved using standard linear regression techniques by defining an appropriate feature transformation $\phi(x)$? (Select all that apply)

Answer: (A, B, D)

Solution. The test in every case is the same: after applying the transformation, does the prediction become a linear combination of the unknown weights? If yes, the normal equations apply unchanged.

(A) $y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$ — **yes.** Take $\phi(x) = [1,\ x,\ x^2,\ x^3]^\top$. The prediction is $\mathbf{w}^\top \phi(x)$, linear in $\mathbf{w}$.

(B) $y = w_0 + w_1 \sin(x) + w_2 \cos(x)$ — **yes.** Take $\phi(x) = [1,\ \sin x,\ \cos x]^\top$. The basis functions are wildly non-linear in x, but that is irrelevant: they are evaluated at the observed x values to fill in numeric columns, and the weights multiply those numbers linearly. This is a basis-function model of exactly the same type as (A).

(D) $y = w_0 + w_1(x_1 \times x_2)$ — **yes.** Take $\phi(x_1, x_2) = [1,\ x_1 x_2]^\top$. An interaction term is simply a new column formed by multiplying two existing ones, computed once before fitting begins.

(C) $y = w_0 + \exp(w_1 x)$ — **no.** Here a parameter sits *inside* a non-linear function. No transformation of x can help, because the obstruction involves w_1 rather than x: whatever column $\phi(x)$ produces, w_1 multiplies x inside the exponential rather than multiplying a column from outside. Concretely, $\partial \hat{y}/\partial w_1 = x \exp(w_1 x)$ depends on w_1, so the objective is not quadratic in the weights, the gradient is not linear in them, and there are no normal equations to solve. This model requires non-linear least squares or gradient-based optimisation, and its loss surface need not be convex.

Note. *A useful diagnostic: differentiate the prediction with respect to each weight. If every such derivative is free of* all *the weights, the model is linear in the parameters and OLS applies. In (A), (B) and (D) the derivatives are* 1, x, $\sin x$, $x_1 x_2$ *and so on — all pure functions of the data. In (C) the derivative still contains* w_1, *which is the signature of a genuinely non-linear model. Note that* $y = w_0 \exp(w_1 x)$ can *be linearised by taking logarithms, but* $y = w_0 + \exp(w_1 x)$ *cannot, because the additive* w_0 *blocks the log from separating the terms.*

Q5. Suppose you are training an unregularized Ordinary Least Squares (OLS) regression model. If you scale a specific feature x_1 in your dataset by

Back to 1st Page

multiplying it by a constant $c > 0$ (so $x_1^{\text{new}} = c \cdot x_1$), how will the corresponding newly estimated coefficient $\hat{\beta}_1^{\text{new}}$ change compared to the original $\hat{\beta}_1$?

Answer: (B)

Solution. The fitted values must be reproducible under the new parametrisation, since the model can represent exactly the same set of functions either way. For the contribution of this feature to be unchanged,

$$\hat{\beta}_1^{\text{new}} x_1^{\text{new}} = \hat{\beta}_1 x_1 \implies \hat{\beta}_1^{\text{new}} (c\, x_1) = \hat{\beta}_1 x_1 \implies \hat{\beta}_1^{\text{new}} = \frac{\hat{\beta}_1}{c}$$

The coefficient moves *inversely* to the feature. Multiplying a feature by 100 divides its coefficient by 100, so that the product — the only thing the prediction sees — is preserved.

This is the diagonal case of a general result. Replacing X by XM for any invertible M gives the new solution $M^{-1}\mathbf{w}$, and since $(XM)(M^{-1}\mathbf{w}) = X\mathbf{w}$, the fitted values are identical. Here $M = \text{diag}(1, \ldots, c, \ldots, 1)$, whose inverse divides the corresponding entry by c.

(A) — incorrect. Multiplying by c has the scaling the right size but the wrong direction; it would multiply this feature's contribution by c^2.

(C) — incorrect, and the most important distractor. OLS *is* scale-invariant, but in its *predictions*, not in its coefficients. The fitted values, residuals, training error and R^2 are all unchanged; the coefficients are precisely the part of the model that absorbs the rescaling. Indeed the coefficients must change in order for the predictions to stay fixed — the invariance of the one is produced by the variation of the other.

(D) — incorrect. Quadratic scaling arises for *variances*, where $\text{Var}[aX] = a^2\text{Var}[X]$, and for the entries of $X^\top X$, which involve products of feature values. A regression coefficient is a rate of change and scales linearly.

Note. *The invariance holds only for* unregularized *OLS, as the question is careful to state. Ridge and lasso penalise $\|\mathbf{w}\|$ directly, so a coefficient inflated by a unit choice absorbs a correspondingly larger share of the penalty, and an arbitrary decision about metres versus centimetres ends up determining which features get shrunk. This is why standardising the features before regularising is not optional. It is also why raw coefficient magnitudes should never be read as "importance": comparing $\hat{\beta}_1$ against $\hat{\beta}_2$ is comparing quantities in different units.*

Q6. Suppose you are fitting a linear regression model but you use a standard, non-augmented design matrix $X \in \mathbb{R}^{N \times D}$ and solve for $w = (X^\top X)^{-1} X^\top y$. What geometric restriction are you forcing upon the resulting prediction

Back to 1st Page

hyperplane?

Answer: (C)

Solution. Without a bias column the model is

$$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + w_D x_D$$

with no constant term. Evaluating at $\mathbf{x} = \mathbf{0}$ gives $\hat{y} = 0$ for every possible choice of $\mathbf{w}$. The fitted hyperplane is therefore constrained to pass through the origin: it may take any orientation, but its position is pinned. In $D = 1$ this is the difference between fitting $\hat{y} = wx$, a line through the origin, and $\hat{y} = w_0 + w_1 x$, a line free to sit anywhere.

Equivalently, the intercept has been forced to zero rather than estimated. The model can still be a perfectly reasonable choice when theory demands a zero intercept, but imposing it when the data does not support it introduces bias that no amount of data will remove.

(A) — incorrect. The residual *vector* is orthogonal to the column space of X, which is a property of every least-squares fit, augmented or not, and follows directly from $X^\top(\mathbf{y} - X\mathbf{w}) = \mathbf{0}$. It says nothing about the hyperplane's geometry in feature space, and the prediction hyperplane is certainly not orthogonal to $\mathbf{y}$.

(B) — incorrect. A zero-slope hyperplane would require $\mathbf{w} = \mathbf{0}$, meaning the features are ignored entirely. Removing the intercept does the opposite: it constrains the *position* while leaving all D slopes free.

(D) — incorrect. Interpolation of all N points would require zero residuals, which happens only when $\mathbf{y}$ already lies in the column space of X — typically when $N \leq \text{rank}(X)$. Removing a column makes this *less* likely, not more, since it shrinks the space available to fit.

Note. *A concrete consequence: with an intercept present, applying $X^\top(\mathbf{y} - X\mathbf{w}) = \mathbf{0}$ to the ones column yields $\sum_i r_i = 0$, so the residuals must sum to zero. Drop the intercept and this guarantee disappears. Fitting $\hat{y} = wx$ to $\{(-1, 1), (2, -5), (3, 5)\}$ gives $w = 4/14$ with residuals summing to $-1/7$, not zero — orthogonality to the columns that are* present *still holds, but the zero-sum property was a consequence of a column that is now missing.*

Q7. Let $X \in \mathbb{R}^{N \times D}$ be a design matrix without a bias column. You create an augmented design matrix X_{aug} by appending a column of ones. Which of the following statements are mathematically true? (Select all that apply)

Answer: (A, B, C)

Solution. **(A) — true.** Appending one column to an $N \times D$ matrix leaves the row count untouched and increases the column count by one,

Back to 1st Page

giving $N \times (D+1)$. The number of samples is unaffected by adding a feature.

(B) — true. The rank of any matrix is bounded by its smaller dimension:

$$\operatorname{rank}(X_{\text{aug}}) \leq \min\left(N,\ D+1\right)$$

When $N \geq D+1$ this bound is $D+1$, which is therefore the maximum attainable. It is achieved exactly when the $D+1$ columns are linearly independent — the condition under which $X_{\text{aug}}^{\top} X_{\text{aug}}$ is invertible and the closed form is available.

(C) — true, and the most substantive option. Suppose one column of the original X is constant with every entry equal to 5. Call it $\mathbf{c}$, and let $\mathbf{1}$ be the appended ones column. Then

$$\mathbf{c} = 5 \cdot \mathbf{1}$$

so the two columns are scalar multiples of one another and hence linearly dependent. The columns of X_{aug} are therefore not independent, giving $\operatorname{rank}(X_{\text{aug}}) < D+1$. Since $\operatorname{rank}(X_{\text{aug}}^{\top} X_{\text{aug}}) = \operatorname{rank}(X_{\text{aug}})$ and $X_{\text{aug}}^{\top} X_{\text{aug}}$ is $(D+1) \times (D+1)$, that matrix is rank-deficient and hence singular. The closed form breaks down.

The failure has a clear interpretation. The intercept and the constant feature are competing to explain the same thing, and infinitely many splits between them produce identical predictions: if (b, w_c) is a solution then so is $(b+5t,\ w_c - t)$ for every t. The fitted values $\hat{y}$ remain unique — it is only the coefficients that are indeterminate.

(D) — false. The target vector is untouched by augmentation. Adding an entry to $\mathbf{y}$ would create a shape mismatch: $X_{\text{aug}}\mathbf{w}_{\text{aug}}$ has N entries, so it can only be compared against a $\mathbf{y}$ with N entries. Augmentation adds a *column* to the inputs and an entry to the *weights*, never anything to the targets.

Note. *Option (C) is the "dummy variable trap" in its simplest form. It appears most often with one-hot encoding: if a categorical variable with k levels is encoded as k indicator columns, those columns sum to the all-ones vector and so duplicate the intercept exactly. The standard fix is to drop one level, leaving $k-1$ columns and treating the dropped level as the baseline absorbed into the intercept. Ridge regression sidesteps the problem entirely, since $X^{\top}X + \lambda I$ is invertible for every $\lambda > 0$ — though it resolves the indeterminacy by splitting the effect evenly rather than by identifying a "correct" split, because no such split exists.*

Q8. You fit a linear regression model to predict a house's price (y). One of the features is a binary variable x_{pool}, where $x_{\text{pool}} = 1$ if the house has a pool,

Back to 1st Page

and 0 otherwise. What is the interpretation of the estimated coefficient $\hat{\beta}_{\text{pool}}$?

Answer: (B)

Solution. Separate the pool term from the rest of the model:

$$\hat{y} = \hat{\beta}_0 + \hat{\beta}_{\text{pool}} x_{\text{pool}} + \sum_j \hat{\beta}_j x_j$$

Compare two houses identical in every other feature, one with a pool and one without:

$$\hat{y}_1 = \hat{\beta}_0 + \hat{\beta}_{\text{pool}} + \sum_j \hat{\beta}_j x_j, \qquad \hat{y}_0 = \hat{\beta}_0 + \sum_j \hat{\beta}_j x_j$$

Subtracting, every shared term cancels:

$$\hat{y}_1 - \hat{y}_0 = \hat{\beta}_{\text{pool}}$$

This is exactly option (B). For a binary feature, a "one-unit increase" spans the entire range of the variable, so the usual per-unit reading and the difference-between-groups reading coincide.

(A) — incorrect. This confuses a *difference* with a *level*. Averages are absolute amounts on the price scale; $\hat{\beta}_{\text{pool}}$ is a gap between two predictions. Note also that the raw average price of pool houses minus the raw average of non-pool houses generally does *not* equal $\hat{\beta}_{\text{pool}}$: houses with pools tend to be larger and better located, so that raw difference bundles in size and location effects. The regression coefficient strips them out by holding the other features fixed. The two coincide only if pool ownership is uncorrelated with every other feature in the model.

(C) — incorrect. This runs the model backwards. Here price is the target and pool status an input; option (C) describes $P(\text{pool} \mid \text{price})$, which would require pool status as the target — a classification problem calling for logistic regression. A regression coefficient is not a probability and is not confined to $[0, 1]$.

(D) — incorrect. In this model the pool effect is additive and constant in absolute terms: it adds the same $\hat{\beta}_{\text{pool}}$ to a modest house and to a luxury one. As a percentage those are entirely different, so the coefficient cannot be a percentage.

Note. *Option (D) becomes correct under a small change of model. Regressing* $\log y$ *instead of* y *gives* $\log \hat{y}_1 - \log \hat{y}_0 = \hat{\beta}_{pool}$*, hence* $\hat{y}_1/\hat{y}_0 = e^{\hat{\beta}_{pool}}$*, and for small coefficients* $e^{\beta} \approx 1 + \beta$*, so* $\hat{\beta}_{pool}$ *reads as roughly a* $100\hat{\beta}_{pool}$ *percent change. Log-transformed targets are common in price modelling*

Back to 1st Page

for exactly this reason. Note also the deliberate restraint in (B)'s wording: "expected difference", not "the effect of building a pool". If a relevant feature is missing — plot size, say — then $\hat{\beta}_{pool}$ absorbs part of its influence. "All other features held constant" means the ones in the model*, not everything that matters.*

Q9. In a multiple linear regression model, you discover that two features, x_1 and x_2, are highly correlated (multicollinearity). How does this impact the interpretation of their respective coefficients, β_1 and β_2? (Select all that apply)

Answer: (A, C)

Solution. **(A) — true.** The partial-effect interpretation asks what happens to $\hat{y}$ when x_1 moves by one unit and x_2 stays fixed. When the two features move together in the data, such an observation is rare or absent, so the coefficient describes a comparison the dataset barely supports. The model will happily report a number, but it is extrapolating into a region of feature space it has almost never seen. This is a problem of *interpretation*, and it persists even when the estimates happen to be numerically stable.

(C) — true. Correlated columns make $X^\top X$ nearly singular: one eigenvalue approaches zero, so the loss surface becomes a long flat valley rather than a well-defined bowl. Many quite different $(\hat{\beta}_1, \hat{\beta}_2)$ pairs give almost the same fit, and the split between them is decided by noise. In practice, small changes to the data — dropping a few observations, or resampling — can swing the coefficients substantially, and the standard errors, which scale with $(X^\top X)^{-1}$, become large. Coefficients of the wrong sign are a common symptom.

(B) — false. Two separate errors are packed into this option. First, unless the correlation is *perfect*, $X^\top X$ remains invertible and the closed form returns a unique answer; ill-conditioning is a matter of degree, not a breakdown. Second, "fail to converge" misdescribes OLS, which is solved directly rather than iteratively — there is nothing to converge. Even under perfect collinearity a minimiser still exists; it is merely non-unique, and the fitted values $\hat{y}$ remain uniquely determined.

(D) — false. Nothing in the OLS objective shrinks anything. Squared-error loss minimises residuals and has no preference for small coefficients; if anything, collinearity tends to produce large coefficients of opposite sign that nearly cancel. Automatic shrinkage is what *ridge* regression adds via its $\lambda\|\mathbf{w}\|_2^2$ term, which is a deliberate modification of the objective and not something plain OLS does on its own.

Note. *The distinction between (A) and (C) is worth keeping sharp, since they describe different failures. Option (C) concerns the* estimates*: they are unstable and imprecise. Option (A) concerns the* estimand*: even a*

Back to 1st Page

perfectly precise partial effect would describe a manipulation the data cannot speak to. Note also what multicollinearity does not *harm — predictive accuracy. If future data carries the same correlation structure, predictions remain fine; it is only the attribution of the effect between x_1 and x_2 that is unreliable. Ridge is the standard remedy precisely because it trades a little bias for a large reduction in this variance.*

VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

Back to 1st Page

3 Loss Functions

Key ideas for this section

A loss function $L(y, \hat{y})$ scores how badly a single prediction misses. Six properties are conventionally demanded of it:

	Property	$L_1 = \lvert y - \hat{y} \rvert$	$L_2 = (y - \hat{y})^2$
(a)	Non-negativity: $L(y, \hat{y}) \geq 0$	✓	✓
(b)	Zero at perfection: $L(y, y) = 0$	✓	✓
(c)	Symmetry: $L(y, \hat{y}) = L(\hat{y}, y)$	✓	✓
(d)	Convexity	✓	✓
(e)	Monotonicity in $\lvert y - \hat{y} \rvert$	✓	✓
(f)	Differentiability	×	✓

1. **The table settles the choice.** L_1 and L_2 agree on five rows out of six. Differentiability is the only point of difference, and it is what makes L_2 the default: a smooth loss admits both a closed form and clean gradient methods.

2. **Squaring amplifies.** An error of 10 contributes 100 under L_2 but only 10 under L_1. This is simultaneously L_2's convenience and its weakness — it is why L_2 is sensitive to outliers and L_1 is robust to them.

3. **Convexity is about minima, not smoothness.** The two are independent properties. $\lvert e \rvert$ is the standard example of a function that is convex yet not differentiable.

4. **Positive constant factors are free.** Multiplying a loss by any $c > 0$ leaves the minimiser exactly where it was, changing only the numerical value of the objective and the size of its gradient.

Q1. In the comparison of the L_1 and L_2 losses, which property do the two losses *not* both satisfy?

Answer: (C)

Solution. Reading down the comparison table, L_1 and L_2 agree on every row except the last. Both are non-negative, both vanish at a perfect prediction, both depend only on the magnitude of the error, both are convex, and both rank a closer prediction as better. The single point of difference is **differentiability**.

Back to 1st Page

The failure is at one point only. The function $|y - \hat{y}|$ has slope -1 to the left of zero and $+1$ to the right, with no consistent value at the join:

$$\frac{d}{de}|e| = \begin{cases} -1 & e < 0 \\ \text{undefined} & e = 0 \\ +1 & e > 0 \end{cases}$$

By contrast e^2 has derivative $2e$ everywhere, including a well-defined value of 0 at the minimum.

Why (A) is the strongest distractor. It is a common misconception that L_1 is not convex, presumably because its graph has a corner. It is in fact perfectly convex: the segment joining any two points on the graph of $|e|$ lies on or above the graph, which is the definition. Convexity and differentiability are logically independent, and $|e|$ is the textbook example of the first without the second.

(B), (D), (E) — all satisfied by both. Symmetry holds because both losses depend on e only through $|e|$, so over- and under-prediction of equal size score equally. Non-negativity holds because absolute values and squares are never negative. Monotonicity holds because both $|e|$ and e^2 are increasing in $|e|$.

Note. *This single row of the table is what determines almost everything downstream. Differentiability is what allows $\nabla J = \mathbf{0}$ to be solved, producing the normal equations and hence the closed form; without it, L_1 regression has no closed-form solution and must be handled by linear programming or by subgradient methods. The same corner, transplanted from the loss to the penalty term, is what gives lasso its ability to drive coefficients to exactly zero — so the property that disqualifies L_1 as a loss is precisely what recommends it as a regulariser.*

Q2. Why is convexity a desirable property of a loss function?

Answer: (B)

Solution. A function is convex when the chord joining any two points on its graph lies on or above the graph. The consequence that matters for optimisation is that such a function **has no local minima other than its global minimum**. There are no valleys to be trapped in and no dependence on where the search begins.

This underwrites both solution methods. For the closed form, setting $\nabla J = \mathbf{0}$ locates a stationary point; convexity is what guarantees that this point is the global minimum rather than a saddle or a local dip, so the normal equations can be trusted. For gradient descent, convexity guarantees that following the negative gradient downhill reaches the true optimum from any initialisation.

Back to 1st Page

(A) — incorrect. Convexity does not imply differentiability, as L_1 demonstrates: it is convex everywhere and non-differentiable at zero. The two properties are independent, and a loss can have either without the other.

(C) — incorrect. Convex functions are typically *un*bounded above; both $|e|$ and e^2 grow without limit as the error grows, which is exactly what monotonicity requires. Boundedness is not among the desirable properties at all.

(D) — incorrect, and the subtlest option. Convexity guarantees that every local minimum is global, but not that there is only *one* such point. The squared-error loss is convex for any design matrix, yet when the columns of X are linearly dependent — collinear features, or more features than samples — the minimum is attained along an entire flat subspace, so infinitely many $\mathbf{w}$ achieve it. Uniqueness requires *strict* convexity, which needs $X^\top X$ to be positive definite rather than merely positive semi-definite.

Note. *The distinction in (D) is worth carrying forward. Convexity says: whatever minimum you find is the best one. Strict convexity says additionally: there is only one. Least squares always has the first property and has the second only when X has full column rank. Ridge regression restores strict convexity for any $\lambda > 0$, since $X^\top X + \lambda I$ is positive definite regardless of X — which is why ridge always returns a unique answer even when OLS cannot.*

Q3. A fit produces residuals of magnitude $1, 1, 1, 1$ and 10. What share of the total loss does the single large residual account for under L_1 and under L_2 respectively?

Answer: (A)

Solution. Compute both totals directly.

Under L_1, the loss is the sum of absolute errors:

$$L_1 = 1 + 1 + 1 + 1 + 10 = 14, \qquad \text{outlier share} = \frac{10}{14} \approx 0.714 = 71.4\%$$

Under L_2, the loss is the sum of squared errors:

$$L_2 = 1^2{+}1^2{+}1^2{+}1^2{+}10^2 = 4{+}100 = 104, \qquad \text{outlier share} = \frac{100}{104} \approx 0.962 = 96.2\%$$

The single bad point already dominates under L_1, but under L_2 it commands essentially the entire objective. The four well-fitted points contribute under 4% between them, so the optimiser has almost no incentive

Back to 1st Page

to keep them well fitted — the fitted line will be dragged toward the outlier at their expense.

(B) — **incorrect.** The two figures are the right ones but assigned to the wrong losses. Squaring is what concentrates the loss on the largest error, so the higher share must belong to L_2.

(C) — **incorrect.** Equal shares would require every residual to contribute equally, which happens only when all five have the same magnitude. Here one is ten times the others.

(D) — **incorrect.** These figures correspond to no consistent calculation; both understate the outlier's dominance.

Note. *The general principle: doubling a residual doubles its contribution under L_1 but quadruples it under L_2. Since minimising a sum means attending most to its largest terms, L_2 directs the fit toward whichever points are worst fitted — efficient when large errors genuinely matter, damaging when they arise from data-entry errors or sensor faults. Note that this is a property of the objective itself, not of how it is optimised: no choice of learning rate or solver alters where the minimum sits. Robust alternatives such as the Huber loss address it directly, behaving quadratically for small errors and linearly for large ones, thereby keeping differentiability near zero while capping an outlier's influence.*

Q4. Given that L_1 is not differentiable at zero, why is it still used in practice?

Answer: (C)

Solution. L_1 grows linearly in the error while L_2 grows quadratically, so a large residual contributes far less to the L_1 objective. As the previous question showed numerically, an outlier taking 96% of the L_2 loss takes only 71% of the L_1 loss. The fitted model is correspondingly less distorted by a handful of extreme points, and this **robustness to outliers** is what keeps L_1 in use despite the optimisation inconvenience.

The trade is explicit: a smooth loss surface is exchanged for a fit that cannot be hijacked by a few bad observations. Which side is preferable depends on whether large errors reflect genuinely important misses or contaminated data.

(A) — **incorrect.** If anything the reverse. L_2's gradient shrinks smoothly to zero near the optimum, giving naturally decaying steps; the L_1 subgradient has constant magnitude 1 regardless of proximity, so the iterates tend to oscillate about the minimum unless the step size is decayed deliberately.

Back to 1st Page

(B) — incorrect, and directly contradicted by the premise. L_1 has *no* closed-form solution precisely because it is not differentiable: there is no gradient to set to zero, hence no normal equations. Minimising it is typically cast as a linear program. The clean closed form $\mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$ belongs to L_2 alone.

(D) — incorrect. Both losses are convex; this is one of the five rows on which the table shows agreement.

Note. *Minimising L_1 error is sometimes called* least absolute deviations, *and it has an appealing statistical reading. Where least squares fits the conditional* mean *of y given $\mathbf{x}$, least absolute deviations fits the conditional* median *— and the median's well-known resistance to outliers is exactly the robustness described above. The generalisation to other quantiles gives quantile regression, which uses an asymmetric variant of the absolute loss.*

Q5. What exactly goes wrong with the L_1 loss at zero error?

Answer: (B)

Solution. The graph of $|e|$ is a V with its vertex at $e = 0$. Approaching from the left the slope is constantly -1; approaching from the right it is constantly $+1$. The one-sided limits disagree, so no derivative exists at the corner:

$$\lim_{h \to 0^-} \frac{|h| - 0}{h} = -1, \qquad \lim_{h \to 0^+} \frac{|h| - 0}{h} = +1$$

The derivative therefore **jumps from -1 to $+1$ with no value at the corner**, exactly as option (B) states.

What makes this genuinely awkward rather than a technicality is *where* the corner sits: at zero error, which is precisely the point the optimiser is trying to reach. Gradient methods stumble at the one place we most want them to land. Away from zero the gradient is perfectly well defined but carries constant magnitude 1, so it gives no indication of how close the minimum is — the steps do not shrink on approach, and the iterates oscillate across the corner instead of settling into it.

(A) — incorrect. The derivative is never zero anywhere on $|e|$; its magnitude is 1 on both sides. Vanishing gradients near the optimum are a feature of L_2, where $\frac{d}{de} e^2 = 2e \to 0$ as $e \to 0$, and that is what allows fixed-step gradient descent to converge smoothly there.

(C) — incorrect. The slopes are ± 1, both finite. Blow-up of the derivative is a different pathology altogether, seen in functions such as $\sqrt{|e|}$ at the origin.

Back to 1st Page

(D) — incorrect on both counts. The derivative is *not* well defined at zero, and the loss does *not* stop being convex — $|e|$ is convex on all of $\mathbb{R}$, corner included.

Note. *The standard workaround is the* subgradient*. At the corner, any value in $[-1, +1]$ serves as a valid subgradient, and subgradient descent simply picks one, conventionally 0. This converges, but more slowly than gradient descent on a smooth loss and without a monotonically decreasing objective. An alternative is to smooth the corner, replacing $|e|$ with $\sqrt{e^2 + \delta^2}$ for small δ, or with the Huber loss — both restore differentiability while preserving the linear growth that gives robustness.*

Q6. Why can we not simply use $\sum_i (y^{(i)} - \hat{y}^{(i)})$ as the loss?

Answer: (B)

Solution. The errors $y^{(i)} - \hat{y}^{(i)}$ are *signed*: points above the fitted surface give positive values and points below give negative ones. Summing them lets these cancel, so a model can score zero while fitting nothing.

A minimal illustration: predictions missing by $+100$ on one point and -100 on another sum to 0, the same score achieved by a model that predicts both points exactly. The objective cannot distinguish a perfect fit from a catastrophic one.

Worse still, the sum is not bounded below. It is linear in $\mathbf{w}$, so it can be driven to $-\infty$ by pushing predictions arbitrarily far above the targets. A minimisation problem with no finite minimum has no solution to find. Removing the sign — by absolute value or by squaring — is exactly what repairs this, and it is what the non-negativity requirement on a loss is really enforcing.

(A) — incorrect. The signed sum is perfectly differentiable. It is linear in $\mathbf{w}$, hence smooth everywhere, with the constant gradient $-X^\top \mathbf{1}$. Differentiability is not the problem here.

(C) — incorrect. A linear function is convex — and concave simultaneously, since it satisfies both defining inequalities with equality. So the signed sum passes the convexity test. Convexity is necessary for a well-behaved loss but plainly not sufficient, and this option is a useful reminder of that.

(D) — incorrect as the *reason*, though the observation is true. Any sum over n terms grows with n, including the perfectly serviceable L_1 and L_2 losses. This is why objectives are conventionally averaged by a factor $1/n$ — a cosmetic fix that makes the value comparable across dataset sizes. It has nothing to do with why the signed sum fails.

Back to 1st Page

Note. *Note that squaring and taking absolute values are not the only ways to remove the sign, but they are the natural ones. Any even function of e that is increasing in $|e|$ would serve — e^4 and $|e|^3$ both qualify. The choice among them is settled by convenience and by how aggressively large errors should be punished, not by whether they are valid.*

Q7. What does the monotonicity requirement on a loss function state?

Answer: (C)

Solution. Monotonicity requires the loss to respect the ordering of errors. Formally, for a fixed true value y,

$$|y - \hat{y}_1| < |y - \hat{y}_2| \implies L(y, \hat{y}_1) < L(y, \hat{y}_2)$$

A prediction that is closer to the truth must score better. Without this, the loss could rank a worse prediction as preferable, and minimising it would drive the model in the wrong direction — the objective would no longer encode the thing we actually want.

Both L_1 and L_2 satisfy it, since $|e|$ and e^2 are both increasing functions of $|e|$.

(A) — incorrect, and a category error. This describes the behaviour of the loss *during training*, which is a property of the optimisation procedure rather than of the loss function. Monotonicity is a statement about L as a mathematical object, evaluated independently of any training run. (The claim is also false as stated — gradient descent with too large a step size, or SGD with a constant one, produces a loss curve that goes up as well as down.)

(B) — incorrect. This is *symmetry*: $L(y, \hat{y}) = L(\hat{y}, y)$, meaning the loss depends on the size of the error but not its direction. It is a separate requirement, and Q9 tests it directly.

(D) — incorrect, though tempting. Unbounded growth is a stronger and different condition. Monotonicity requires only that the loss keep increasing; it does not require it to increase without limit. A loss such as $1 - e^{-e^2}$ is strictly increasing in $|e|$ and so monotone, yet it saturates at 1. Such saturating losses are used deliberately in robust statistics precisely because they cap an outlier's influence.

Note. *Monotonicity, symmetry and zero-at-perfection together pin down the qualitative shape of a loss: a function of $|e|$ alone, equal to zero at the origin and rising from there. What remains open is* how fast *it rises, and that single remaining choice is what distinguishes L_1 from L_2 from Huber. Note also that symmetry, unlike the other properties, is genuinely optional in applications: when under-forecasting demand costs far more than over-forecasting it, an asymmetric loss is the correct modelling choice.*

Back to 1st Page

Q8. What is the effect of writing the objective as $J(\mathbf{w}) = \frac{1}{2n}\sum_i (y^{(i)} - \hat{y}^{(i)})^2$ rather than as the plain sum of squares?

Answer: (C)

Solution. Both factors are positive constants, and multiplying an objective by a positive constant rescales the surface vertically without moving its lowest point. If $J_{\text{new}} = cJ_{\text{raw}}$ with $c = 1/(2n) > 0$, then

$$\nabla J_{\text{new}} = c\,\nabla J_{\text{raw}} \implies \nabla J_{\text{new}} = \mathbf{0} \iff \nabla J_{\text{raw}} = \mathbf{0}$$

so the two objectives have exactly the same stationary points and hence **the same minimiser**. The fitted model is identical either way.

What does change is the *magnitude* of the gradient, and with it the range of usable learning rates. The curvature matrix is scaled by c along with everything else, so its largest eigenvalue is scaled by c and the stability condition $\alpha < 1/\lambda_{\max}$ permits a step size $2n$ times larger than before. A learning rate tuned for one convention will misbehave under the other, even though both describe the same optimisation problem.

The two factors serve distinct purposes. The $1/n$ converts a sum into a mean, so the objective does not grow simply because the dataset is large and losses remain comparable across datasets of different sizes. The $1/2$ is pure algebraic convenience: differentiating a square produces a factor of 2, and the $\frac{1}{2}$ cancels it, leaving clean expressions such as $\nabla J = \frac{1}{n}X^\top(X\mathbf{w} - \mathbf{y})$.

(A) — incorrect. The minimiser is unchanged, as shown above; no rescaling of $\mathbf{w}$ afterwards is needed or appropriate.

(B) — incorrect. The raw sum of squares is already convex — a sum of convex functions is convex, and multiplying by a positive constant preserves that. Convexity is not something the factor supplies.

(D) — incorrect, and the closest wrong answer. It is correct that the solution is unaffected, but "no effect at all" overstates the case. The gradient magnitude changes, so in practice the choice of α must change with it. Anyone who has moved code between the two conventions without retuning the learning rate has met this.

Note. *This is a specific instance of a general fact from the contour geometry: scaling every coefficient of a quadratic by the same positive factor leaves its shape and orientation untouched, and therefore leaves the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ untouched as well. The problem is neither easier nor harder to optimise; only the numerical value of a suitable α shifts. Contrast this with* feature *scaling, which changes the coefficients unequally, genuinely alters κ, and so genuinely alters the iteration count.*

Back to 1st Page

Q9. A loss that penalises predicting 10 when the truth is 8 exactly as much as predicting 6 when the truth is 8 is exhibiting which property?

Answer: (B)

Solution. The two predictions have errors $\hat{y} - y = +2$ and -2 respectively: equal in magnitude, opposite in direction. A loss that scores them identically depends on the error only through its size, ignoring its sign. This is **symmetry**, written

$$L(y, \hat{y}) = L(\hat{y}, y)$$

Both standard losses have it, since $|+2| = |-2| = 2$ and $(+2)^2 = (-2)^2 = 4$.

(A) — incorrect. Monotonicity compares errors of *different* magnitudes and requires the smaller one to score better. Here the magnitudes are equal, so monotonicity has nothing to say; it neither forces nor forbids the two scores to agree.

(C) — incorrect. Convexity constrains the curvature of the loss — how it behaves between points — and is entirely silent on whether it treats the two sides of zero alike. Asymmetric convex losses exist and are common: the quantile (pinball) loss, which penalises over- and under-prediction by different multiples, is convex and deliberately not symmetric.

(D) — incorrect. Zero at perfection concerns the single point $\hat{y} = y$, requiring $L(y, y) = 0$. Neither prediction here is exact, so the property is not in play.

Note. *Symmetry is the one property on the list that is frequently* unwanted. *In inventory planning, a stockout typically costs more than an equivalent overstock; in medical screening, a missed diagnosis is not equivalent to a false alarm; in energy forecasting, under-generation and over-generation have very different consequences. Asymmetric losses handle these directly — the pinball loss uses multipliers τ and $1 - \tau$ on positive and negative errors — and fitting them yields conditional quantiles rather than the conditional mean. Symmetry is best regarded as the default assumption behind ordinary least squares, not as a universal requirement.*

Back to 1st Page

4 Closed Form Solution

Key ideas for this section

1. **The derivation.** Expanding $L(\mathbf{w}) = (\mathbf{y} - X\mathbf{w})^\top(\mathbf{y} - X\mathbf{w})$ gives $\mathbf{y}^\top\mathbf{y} - 2\mathbf{y}^\top X\mathbf{w} + \mathbf{w}^\top(X^\top X)\mathbf{w}$, where the two cross terms merge because each is a scalar. Differentiating with $\nabla_{\mathbf{w}}(\mathbf{a}^\top\mathbf{w}) = \mathbf{a}$ and $\nabla_{\mathbf{w}}(\mathbf{w}^\top A\mathbf{w}) = 2A\mathbf{w}$ for symmetric A, then setting the result to zero, gives the *normal equations*:

$$-2X^\top\mathbf{y} + 2X^\top X\mathbf{w} = \mathbf{0} \iff X^\top X\mathbf{w} = X^\top\mathbf{y} \implies \hat{\mathbf{w}} = (X^\top X)^{-1}X^\top\mathbf{y}$$

2. **Orthogonality.** The normal equations say exactly $X^\top\mathbf{r} = \mathbf{0}$ for $\mathbf{r} = \mathbf{y} - X\hat{\mathbf{w}}$: the residual is orthogonal to every column of X. This holds always. Consequences that require a *particular* column — such as the residuals summing to zero — hold only when that column is present.

3. **The hat matrix.** $H = X(X^\top X)^{-1}X^\top$ satisfies $\hat{\mathbf{y}} = H\mathbf{y}$. It is symmetric and idempotent, its eigenvalues are 0 or 1, and $\mathrm{tr}(H) = \mathrm{rank}(X)$. Its diagonal entries h_{ii} are the leverages. The residual maker is $M = I - H$.

4. **Cost.** Forming $X^\top X$ costs $O(nd^2)$ and solving the $d \times d$ system costs $O(d^3)$, so the total is $O(nd^2 + d^3)$: linear in n, cubic in d.

5. **Existence versus uniqueness.** A minimiser always exists. It is unique precisely when X has full column rank, which requires $n \geq d$ but is not implied by it.

Q1. To derive the normal equations for Ordinary Least Squares, we minimize the residual sum of squares $L(w) = (y - Xw)^\top(y - Xw)$. Which of the following represents the correct gradient $\nabla_w L(w)$ set to zero?

Answer: (B)

Solution. Expand the objective, treating $X \in \mathbb{R}^{n\times d}$, $\mathbf{w} \in \mathbb{R}^d$ and $\mathbf{y} \in \mathbb{R}^n$:

$$L(\mathbf{w}) = \mathbf{y}^\top\mathbf{y} - \mathbf{y}^\top X\mathbf{w} - \mathbf{w}^\top X^\top\mathbf{y} + \mathbf{w}^\top X^\top X\mathbf{w}$$

The two middle terms are transposes of one another, and each is a 1×1 scalar, so they are equal and combine:

$$L(\mathbf{w}) = \mathbf{y}^\top\mathbf{y} - 2\,\mathbf{y}^\top X\mathbf{w} + \mathbf{w}^\top(X^\top X)\mathbf{w}$$

Differentiate term by term. The first is constant in $\mathbf{w}$. The second is linear, of the form $\mathbf{a}^\top\mathbf{w}$ with $\mathbf{a} = -2X^\top\mathbf{y}$, so it contributes $-2X^\top\mathbf{y}$. The third is a quadratic form $\mathbf{w}^\top A\mathbf{w}$ with $A = X^\top X$, which is symmetric, so it contributes $(A + A^\top)\mathbf{w} = 2X^\top X\mathbf{w}$. Hence

$$\nabla_{\mathbf{w}} L = -2X^\top\mathbf{y} + 2X^\top X\mathbf{w} = \mathbf{0}$$

Back to 1st Page

which is option (B). Dividing by 2 and rearranging gives the normal equations $X^\top X\mathbf{w} = X^\top \mathbf{y}$.

Shape check on the distractors. The gradient must be $d \times 1$, matching $\mathbf{w}$. Two options fail before any calculus is needed.

(A) — invalid. The term $Xy^\top$ multiplies $(n \times d)$ by $(1 \times n)$; the inner dimensions d and 1 do not match, so the product does not exist.

(C) — invalid. Here Xy multiplies $(n \times d)$ by $(n \times 1)$, undefined unless $d = n$. Likewise $XX^\top w$ multiplies the $n \times n$ matrix $XX^\top$ by a $d \times 1$ vector. Note that $XX^\top$ is a legitimate matrix, just the wrong one — it is the $n \times n$ Gram matrix that appears in the dual formulation, not in the primal.

(D) — correct shapes, wrong coefficient. Both $2X^\top y$ and $X^\top Xw$ are $d \times 1$, so this survives the shape check. It fails on the factor of 2: differentiating $\mathbf{w}^\top A\mathbf{w}$ produces $2A\mathbf{w}$, because $\mathbf{w}$ appears twice and the product rule contributes a term for each occurrence. Option (D) accounts for only one, and solving it would give $\mathbf{w} = 2(X^\top X)^{-1}X^\top \mathbf{y}$ — exactly twice the correct answer.

Note. *The one-dimensional analogue exposes (D) immediately: with $d = n = 1$ the objective is $(y - xw)^2$, whose derivative is $-2x(y - xw) = -2xy + 2x^2 w$. Both terms carry the factor 2. This is the same trap as the identity $\nabla_{\mathbf{x}}(\mathbf{x}^\top W\mathbf{x}) = 2W\mathbf{x}$, where writing $W\mathbf{x}$ instead is the standard error. Note also that setting the gradient to zero suffices here only because L is convex — $X^\top X$ is positive semi-definite, so the stationary point is a global minimum rather than a saddle.*

Q2. Consider a dataset with a single feature and no intercept, where the feature matrix is $X = \begin{bmatrix}1\\2\end{bmatrix}$ and the target vector is $y = \begin{bmatrix}3\\4\end{bmatrix}$. What is the exact closed-form OLS estimate for the weight w?

Answer: (A)

Solution. With a single feature and no intercept, X is 2×1, so $X^\top X$ is a 1×1 matrix — an ordinary number — and the inverse is just its reciprocal.

Step 1. Compute $X^\top X$:

$$X^\top X = \begin{bmatrix}1 & 2\end{bmatrix}\begin{bmatrix}1\\2\end{bmatrix} = 1^2 + 2^2 = 5$$

Step 2. Compute $X^\top y$:

$$X^\top y = \begin{bmatrix}1 & 2\end{bmatrix}\begin{bmatrix}3\\4\end{bmatrix} = (1)(3) + (2)(4) = 3 + 8 = 11$$

Back to 1st Page

Step 3. Apply the closed form:

$$\hat{w} = (X^\top X)^{-1} X^\top y = \frac{11}{5} = 2.2$$

In general, regression through the origin with one feature reduces to

$$\hat{w} = \frac{\sum_i x_i y_i}{\sum_i x_i^2}$$

a ratio that weights each observation by its own x_i. Points far from the origin therefore exert more influence, which is the one-dimensional shadow of leverage.

Verification. The fitted values are $\hat{\mathbf{y}} = \frac{11}{5}\begin{bmatrix}1\\2\end{bmatrix} = \begin{bmatrix}2.2\\4.4\end{bmatrix}$, giving residuals $\mathbf{r} = \begin{bmatrix}0.8\\-0.4\end{bmatrix}$. Checking orthogonality against the single column:

$$X^\top \mathbf{r} = (1)(0.8) + (2)(-0.4) = 0.8 - 0.8 = 0 \quad \checkmark$$

(B) — incorrect. This inverts the ratio, computing $X^\top y$ in the denominator. **(C) — incorrect.** This is $(3+4)/5$, summing the targets rather than weighting each by its feature value. **(D) — incorrect.** This is $y_2/x_2 = 4/2$, the line through the second point alone, ignoring the first.

Note. *Note that the residuals here sum to $0.8 - 0.4 = 0.4 \neq 0$. Nothing has gone wrong: this model has no intercept column, and the zero-sum property is a consequence of that column rather than of least squares in general. The next question turns on exactly this point, and this dataset serves as its counterexample.*

Q3. Let $r = y - X\hat{w}$ be the residual vector from an OLS regression where $\hat{w}$ is the optimal closed-form solution. Which of the following statements are *always* true regardless of whether the model has an intercept? (Select all that apply)

Answer: (A, B)

Solution. (A) $X^\top r = 0$ **— always true.** This is a restatement of the normal equations. From $X^\top X\hat{\mathbf{w}} = X^\top \mathbf{y}$,

$$X^\top \mathbf{r} = X^\top(\mathbf{y} - X\hat{\mathbf{w}}) = X^\top \mathbf{y} - X^\top X\hat{\mathbf{w}} = \mathbf{0}$$

Geometrically, the residual is orthogonal to every column of X, hence to the whole column space. This is the defining property of the optimum and depends on no assumption about which columns are present.

Back to 1st Page

(B) $\hat{y}^\top r = 0$ — **always true.** The fitted vector lies in the column space, $\hat{\mathbf{y}} = X\hat{\mathbf{w}}$, so orthogonality to the columns transfers to it:

$$\hat{\mathbf{y}}^\top \mathbf{r} = (X\hat{\mathbf{w}})^\top \mathbf{r} = \hat{\mathbf{w}}^\top \underbrace{X^\top \mathbf{r}}_{=\mathbf{0}} = 0$$

It follows from (A) with no extra hypotheses.

(C) $\sum_{i=1}^{n} r_i = 0$ — ***not*** **always true.** This is (A) applied to a *specific* column: if X contains a column of ones $\mathbf{1}$, then $\mathbf{1}^\top \mathbf{r} = \sum_i r_i = 0$. Remove the intercept and that column is gone, and with it the guarantee. The previous question supplies the counterexample: fitting $\hat{y} = wx$ to $(1, 3)$ and $(2, 4)$ gives residuals 0.8 and -0.4, summing to $0.4 \neq 0$, even though $X^\top \mathbf{r} = 0$ holds exactly.

(D) $y^\top r = 0$ — **false, and false in an instructive way.** Decompose $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{r}$ and use (B):

$$\mathbf{y}^\top \mathbf{r} = (\hat{\mathbf{y}} + \mathbf{r})^\top \mathbf{r} = \underbrace{\hat{\mathbf{y}}^\top \mathbf{r}}_{=0} + \mathbf{r}^\top \mathbf{r} = \|\mathbf{r}\|^2$$

This vanishes only when $\mathbf{r} = \mathbf{0}$, that is, when the fit is perfect. The residual is orthogonal to the *projection* of $\mathbf{y}$, not to $\mathbf{y}$ itself. On the data above: $\mathbf{y}^\top \mathbf{r} = (3)(0.8) + (4)(-0.4) = 0.8$, which is indeed $\|\mathbf{r}\|^2 = 0.64 + 0.16 = 0.8$.

Note. *The identity in (D) is the engine behind the variance decomposition. Since $\mathbf{y} = \hat{\mathbf{y}} + \mathbf{r}$ with $\hat{\mathbf{y}} \perp \mathbf{r}$, Pythagoras gives $\|\mathbf{y}\|^2 = \|\hat{\mathbf{y}}\|^2 + \|\mathbf{r}\|^2$, which becomes TSS = ESS + RSS once the vectors are centred — and centring is legitimate precisely when an intercept is present. So option (C), far from being a technicality, is the condition under which the familiar R^2 decomposition is valid at all.*

Q4. Let $H = X(X^\top X)^{-1}X^\top$ be the hat matrix (projection matrix) in OLS regression. Which of the following properties accurately describe H? (Select all that apply)

Answer: (A, B, D)

Solution. (A) Idempotent, $H^2 = H$ — true. Multiply out, cancelling the adjacent inverse pair:

$$H^2 = X(X^\top X)^{-1} \underbrace{X^\top X (X^\top X)^{-1}}_{=I} X^\top = X(X^\top X)^{-1}X^\top = H$$

Geometrically: H projects onto the column space, and $H\mathbf{y}$ already lies there, so projecting a second time changes nothing.

(B) Symmetric, $H^\top = H$ — true. Transposing reverses the order of the factors:

$$H^\top = \big(X(X^\top X)^{-1}X^\top\big)^\top = X\big((X^\top X)^{-1}\big)^\top X^\top = X(X^\top X)^{-1}X^\top = H$$

Back to 1st Page

using the fact that $X^\top X$ is symmetric, hence so is its inverse. Symmetry together with idempotence is what makes H an *orthogonal* projection rather than an oblique one.

(D) The diagonal entries h_{ii} are leverages — true. Writing $\hat{y}_i = \sum_j h_{ij} y_j$ and differentiating,

$$h_{ii} = \frac{\partial \hat{y}_i}{\partial y_i}$$

so h_{ii} measures how strongly observation i pulls its own fitted value. Since H is built from X alone, leverage depends only on where a point sits in feature space and can be computed before any y is observed. One always has $0 \le h_{ii} \le 1$, and $\sum_i h_{ii} = \mathrm{tr}(H) = \mathrm{rank}(X)$, giving an average leverage of d/n.

(C) $Hy = r$ — false. H produces the *fitted* vector, not the residual:

$$H\mathbf{y} = X(X^\top X)^{-1} X^\top \mathbf{y} = X\hat{\mathbf{w}} = \hat{\mathbf{y}}$$

The residual comes from the complementary projection:

$$\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}} = (I - H)\mathbf{y}$$

The matrix $M = I - H$ is called the residual maker. It is symmetric and idempotent in its own right, projects onto the orthogonal complement of the column space, and satisfies $HM = 0$ — the algebraic statement that fitted values and residuals are orthogonal.

Note. *The name "hat matrix" is literal: H is what puts the hat on $\mathbf{y}$. Two further facts follow from symmetry plus idempotence. First, every eigenvalue satisfies $\lambda^2 = \lambda$, so each is 0 or 1, and the number of ones equals the rank. Second, $\mathrm{tr}(H) = \mathrm{rank}(X) = d$ for a full-rank design, while $\mathrm{tr}(M) = n - d$ — which is the source of the $n - d$ denominator in the unbiased variance estimate $\hat{\sigma}^2 = \|\mathbf{r}\|^2/(n-d)$. Note also that H is singular whenever $d < n$, since it has $n - d$ zero eigenvalues; a projection deliberately destroys information, which no invertible map can do.*

Q5. Let $X \in \mathbb{R}^{n \times d}$ be a feature matrix with n samples and d features (where $n \gg d$). What is the standard time complexity for computing the closed-form OLS solution $\hat{w} = (X^\top X)^{-1} X^\top y$?

Answer: (C)

Solution. Cost the expression one operation at a time.

VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

Back to 1st Page

Step	Shapes	Cost
Form $X^\top X$	$(d \times n)(n \times d) \to d \times d$	$O(nd^2)$
Form $X^\top y$	$(d \times n)(n \times 1) \to d \times 1$	$O(nd)$
Solve $(X^\top X)\mathbf{w} = X^\top y$	$d \times d$ system	$O(d^3)$

Forming $X^\top X$ requires computing d^2 entries, each an inner product of length n, giving $O(nd^2)$. Solving the resulting square system — by Cholesky, LU or explicit inversion — costs $O(d^3)$. The $O(nd)$ term for $X^\top y$ is dominated by $O(nd^2)$ and is dropped. The total is

$$O(nd^2 + d^3)$$

which is option (C). The two terms carry very different consequences: cost is only *linear* in n, so a million rows is unproblematic, but *cubic* in d, so tens of thousands of features are fatal. This asymmetry is precisely what motivates gradient descent, whose per-iteration cost is only $O(nd)$.

(A) $O(n^2d + d^3)$ — **incorrect, but for an interesting reason.** The term $O(n^2d)$ is the cost of forming $XX^\top$, the $n \times n$ Gram matrix. That is a real computation, appearing in the dual or kernel formulation which is preferred when $d \gg n$ — but it is not what the given expression computes.

(B) $O(n^3+d^3)$ — **incorrect.** An n^3 term would mean inverting an $n \times n$ matrix. No matrix of that size is ever inverted in the primal closed form; the only matrix inverted is the $d \times d$ Gram matrix.

(D) $O(nd + d^2)$ — **incorrect.** These are the costs of a matrix–vector product and of merely reading a $d \times d$ matrix. Both are too optimistic: they omit the d inner products of length n needed for each column of $X^\top X$, and the elimination needed to solve the system.

Note. *Two practical footnotes. First, $X^\top X$ and $X^\top y$ can be accumulated in a single streaming pass over the rows, since both are of fixed size independent of n — so the closed form does not require the entire dataset to be held in memory at once. Second, one should never form $(X^\top X)^{-1}$ explicitly. Solving the system directly costs the same $O(d^3)$ but is numerically far better behaved, and QR or SVD factorisations of X are better still, since building $X^\top X$ squares the condition number of the problem.*

Q6. Consider the OLS closed-form solution $\hat{w} = (X^\top X)^{-1}X^\top y$. Which of the following conditions guarantee that a *unique* solution for $\hat{w}$ exists? (Select all that apply)

Answer: (A, C, D)

Back to 1st Page

Solution. Uniqueness holds exactly when $X^\top X$ is invertible. Options (A), (C) and (D) are three equivalent ways of stating that same condition.

(A) X has full column rank — guarantees uniqueness. Full column rank means $\text{rank}(X) = d$. Since $\text{rank}(X^\top X) = \text{rank}(X)$ and $X^\top X$ is $d \times d$, the Gram matrix has full rank and is therefore invertible.

(D) The columns of X are linearly independent — guarantees uniqueness. This is the definition of full column rank, so it is option (A) restated. Equivalently, $X\mathbf{v} = \mathbf{0}$ has only the trivial solution.

(C) $X^\top X$ is positive definite — guarantees uniqueness. A positive definite matrix has all eigenvalues strictly positive, hence non-zero determinant, hence an inverse. This too is equivalent to the others:

$$\mathbf{v}^\top X^\top X\mathbf{v} = \|X\mathbf{v}\|^2 > 0 \text{ for all } \mathbf{v} \neq \mathbf{0} \iff X\mathbf{v} \neq \mathbf{0} \text{ for all } \mathbf{v} \neq \mathbf{0}$$

which is again linear independence of the columns. Note that $X^\top X$ is *always* positive semi-definite, since $\|X\mathbf{v}\|^2 \geq 0$ unconditionally; the content of (C) lies entirely in the strictness.

(B) $n < d$ strictly — guarantees the *opposite*. Rank is bounded by both dimensions, so

$$\text{rank}(X) \leq \min(n, d) = n < d$$

The columns cannot be independent, $X^\top X$ is singular, and the closed form breaks down. Far from ensuring uniqueness, this condition rules it out: infinitely many $\hat{\mathbf{w}}$ achieve the same minimum loss. The wide-data regime $n < d$ is common in genomics and text, and it is exactly where regularization or the pseudo-inverse becomes necessary.

Note. *Two clarifications worth keeping straight. First, existence is never the issue — a convex quadratic bounded below always attains its minimum, and even when $X^\top X$ is singular the fitted vector $\hat{\mathbf{y}}$ is uniquely determined as the projection of $\mathbf{y}$ onto the column space. It is only the* coordinates $\hat{\mathbf{w}}$ *describing that point which become indeterminate. Second, $n \geq d$ is necessary but not sufficient: a million rows will not save you if two feature columns are duplicates. Ridge regression removes the issue entirely, since $X^\top X + \lambda I$ is positive definite for every $\lambda > 0$, which is why it returns a unique answer even when $n < d$.*

Back to 1st Page

5 Gradient Descent

Key ideas for this section

1. **The update rule.** $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \alpha \nabla J(\mathbf{w}^{(k)})$. The gradient points uphill, so the minus sign turns it downhill. All weights are updated simultaneously from the same $\mathbf{w}^{(k)}$.

2. **On a quadratic, each step is a multiplication.** For $J = \sum_i c_i w_i^2$ the update reduces to $w_i \leftarrow (1 - 2\alpha c_i) w_i$. Convergence needs $|1 - 2\alpha c_i| < 1$ for *every* coordinate, hence $\alpha < 1/c_{\max}$ — the steepest direction caps the step size while the flattest governs the speed, so their ratio κ sets the iteration count.

3. **Learning-rate regimes.** Too small is slow; well chosen is fast; slightly too large converges while oscillating; too large diverges. It is not a monotone slide from slow to broken — there is an optimum in the middle.

4. **Stopping criteria.** Iteration budget, small change in loss, small gradient norm, and validation loss. Each measures something correlated with convergence without being equivalent to it, which is why practical code combines them.

5. **Counting.** An epoch is one full pass over the data. With mini-batch size B it contains $\lceil n/B \rceil$ updates; batch descent gives 1, pure SGD gives n. All three perform the same number of gradient *evaluations* per epoch.

6. **Closed form versus iteration.** The closed form costs $O(nd^2 + d^3)$ and is exact; gradient descent costs $O(nd)$ per iteration and is approximate. Large d, streaming data, or a loss with no closed form all favour iteration.

Q1. Consider the objective function $f(x) = x^2 - 4x$. You are performing standard Gradient Descent to find its minimum. If your starting point is $x_0 = 0$ and your learning rate is $\eta = 0.1$, what is the value of the parameter after one iteration (x_1)?

Answer: (A)

Solution. Step 1 — differentiate.

$$f'(x) = 2x - 4$$

Step 2 — evaluate at the starting point.

$$f'(0) = 2(0) - 4 = -4$$

The gradient is negative, meaning the function is still descending as x increases. We should therefore move to the *right*.

Back to 1st Page

Step 3 — apply the update.

$$x_1 = x_0 - \eta f'(x_0) = 0 - (0.1)(-4) = 0 + 0.4 = 0.4$$

Subtracting a negative gradient adds to the parameter, which is exactly the behaviour we want: the minimum sits at $x^\star = 2$, to the right of the start.

Sanity check via the multiplier. The update in closed form is

$$x \leftarrow x - \eta(2x - 4) = (1 - 2\eta)x + 4\eta = 0.8x + 0.8$$

Its fixed point solves $x = 0.8x + 0.8$, giving $x = 2 = x^\star$ as required, and the error contracts by a factor 0.8 per step.

(B) -0.4 — **the sign trap.** This adds the gradient instead of subtracting it, moving *away* from the minimum. It is the single most common error in this calculation. **(C)** 0.8 — **double the correct step**, as would arise from $\eta = 0.2$. **(D)** 4.0 — **the gradient magnitude itself**, with the learning rate omitted.

Note. *Note that $|1 - 2\eta| < 1$ requires $0 < \eta < 1$ here, so this problem tolerates a wide range of learning rates. The special value $\eta = 0.5$ gives multiplier zero and lands exactly on $x^\star = 2$ in one step — that is Newton's method, since $\eta = 1/f''(x)$. Larger objectives are not so forgiving: for $J = 5w^2$ the same reasoning caps α at 0.2.*

Q2. Which of the following statements about the learning rate η in gradient descent are correct? (Select all that apply)

Answer: (A, C)

Solution. (A) — true. If η exceeds the stability threshold, the per-step multiplier satisfies $|1 - \eta\lambda| > 1$ in at least one direction, so the error is amplified rather than reduced. The parameters spiral outward and the loss increases without bound, typically ending in numerical overflow.

(C) — true. A schedule keeps η large early, when large steps are productive, and shrinks it later so the iterates can settle rather than skip over the minimum. This matters most with stochastic gradients, where the noise floor scales with η: a constant rate leaves the iterates bouncing in a region that never shrinks.

(B) — false, and the most instructive distractor. The first clause is right, the second is not. Small steps merely mean the trajectory follows the local downhill direction closely — and on a non-convex surface the local downhill direction leads to whatever basin you happen to have started in. If anything the implication runs backwards: a very small η makes the algorithm *more* likely to settle into the nearest local minimum, whereas

Back to 1st Page

a larger one can step over narrow basins. Global optimality follows from *convexity*, never from step size.

(D) — false. There is no universal optimal rate. The stability ceiling is $\eta < 2/\lambda_{\max}(H)$, which is a property of the problem's curvature, not a fixed number. If the curvature is large the usable η may be far below 1; if it is tiny, a rate well above 1 can be perfectly stable. In practice η is searched on a logarithmic grid because it matters multiplicatively.

Note. *Option (D) also ignores that the ceiling* moves *during training. On a non-quadratic loss the Hessian changes as you travel, so a rate that is safe at initialisation may destabilise later, and vice versa. This is precisely what learning-rate warmup addresses: early curvature is often large and badly conditioned, so one starts small and ramps up once the trajectory settles.*

Q3. Which of the following are valid stopping criteria for Gradient Descent, and correctly state their potential failure modes? (Select all that apply)

Answer: (A, C, D)

Solution. (A) — true. The gradient vanishes at every stationary point, not only at minima. On a non-convex surface a saddle point also has $\nabla L \approx \mathbf{0}$, so the criterion fires there and reports convergence at a point that is not even a local minimum. Note that this failure mode is absent for least squares, where convexity guarantees the only stationary point is the global minimum — but that is a property of *this* loss, not of the criterion.

(C) — true, and the subtlest option. The update magnitude is

$$\|\theta_t - \theta_{t-1}\| = \eta \, \|\nabla L(\theta_{t-1})\|$$

so it is small when *either* factor is small. If the schedule decays η aggressively, the product falls below ϵ while the gradient is still large, and training halts far from the optimum. The criterion cannot distinguish "we have arrived" from "we have stopped moving".

(D) — true. Early stopping on validation loss is sound practice, but the validation estimate carries its own variance. A small or unrepresentative split gives a noisy curve whose apparent upturn may be sampling noise rather than genuine overfitting, causing a stop that is too early or too late.

(B) — false. Two errors. First, a stalled training loss indicates slow progress, not proximity to the optimum: along a long flat valley the loss barely moves per step while the weights are still far away. Second, and more seriously, nothing about *training* loss can guarantee anything about *generalization*. The point at which training loss plateaus is frequently well past the point where validation loss has begun to rise.

Back to 1st Page

Note. *The three failure modes are usefully contrasted. (A) mistakes a flat point for a minimum, (C) mistakes a small step for a small gradient, and (B) mistakes training performance for test performance. Because each measures a different proxy, production code typically combines them: a maximum iteration budget as a hard cap, a relative tolerance on the gradient, and validation-based early stopping for the model actually kept.*

Q4. Suppose you are training a Linear Regression model on a dataset of 10,000 samples using Mini-Batch Gradient Descent. You choose a batch size of 100. How many total parameter updates will the model perform after exactly 5 epochs?

Answer: (C)

Solution. Step 1 — updates per epoch. An epoch is one full pass through the data, and each mini-batch produces exactly one weight update:

$$\left\lceil \frac{n}{B} \right\rceil = \frac{10{,}000}{100} = 100 \text{ updates per epoch}$$

Step 2 — multiply by the number of epochs.

$$100 \times 5 = 500$$

(A) 5 — **counts epochs**, which would be the update count for *batch* gradient descent, where the whole dataset produces a single step. **(B)** 100 — **one epoch's worth**, forgetting to multiply by 5. **(D)** 50,000 — **the total number of examples processed** (10,000 × 5), which is the update count for *pure SGD*, not for mini-batches of 100.

Note. *The three wrong answers are not arbitrary:* 5, 500 *and* 50,000 *are the update counts for batch, mini-batch and pure SGD respectively on the same data and the same number of epochs. All three do identical total gradient work — 50,000 per-example gradient evaluations — and differ only in how those evaluations are grouped before a step is taken.*

Q5. When comparing Batch Gradient Descent (BGD), Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent (MBGD), which statements are true? (Select all that apply)

Answer: (A, B, D)

Solution. (A) — true. BGD averages over all n examples, so it computes the true gradient of the empirical loss with no sampling noise. The trajectory is smooth and, with a suitable α, the loss decreases monotonically. The cost is that a single step requires touching every example, which is prohibitive when n is in the millions and the data does not fit in memory.

Back to 1st Page

(B) — true. A single-example gradient is an unbiased but high-variance estimate of the full gradient. The resulting jitter is not purely a defect: it lets the iterates jump out of shallow basins that a noiseless method would settle into. This is one reason SGD often generalises better on non-convex problems.

(D) — true. A mini-batch gradient is a matrix operation over B examples at once, which maps directly onto GPU and SIMD hardware. Pure SGD processes one example at a time and leaves that parallelism unused, so it is often slower in wall-clock terms despite doing the same arithmetic.

(C) — false, and reversed. Averaging B independent gradient estimates reduces the variance by a factor of B:

$$\mathrm{Var}\left[\frac{1}{B}\sum_{j=1}^{B}\mathbf{g}_j\right] = \frac{1}{B}\mathrm{Var}[\mathbf{g}]$$

So MBGD has *lower* variance than pure SGD, not higher. Mini-batching sits between the two extremes on every axis — variance, cost per step, and steps per epoch — which is exactly why it is the default in practice.

Note. *The three methods are the endpoints and interior of a single spectrum indexed by B. At $B = n$ you get BGD: one exact step per epoch. At $B = 1$ you get SGD: n noisy steps. Typical values of B between 32 and 512 buy most of SGD's step count while recovering most of BGD's stability and all of the hardware parallelism.*

Q6. In the context of OLS Linear Regression, under what circumstances would you strongly prefer using Gradient Descent over the exact closed-form solution $\hat{w} = (X^\top X)^{-1}X^\top y$? (Select all that apply)

Answer: (A, B)

Solution. (A) — true. The closed form costs $O(nd^2 + d^3)$. With $d > 10^6$ the d^3 term is around 10^{18} operations, and $X^\top X$ alone would be a $10^6 \times 10^6$ matrix requiring terabytes of memory. Gradient descent needs only $O(nd)$ per iteration and never forms $X^\top X$ at all.

(B) — true. The closed form is a batch computation: it assumes a fixed X and y and must be redone from scratch when new data arrives. Gradient descent — and SGD in particular — updates the current weights from each new example as it appears, needing only one observation in memory at a time. This is the natural fit for streaming and online settings.

(C) — false, and argues the opposite. Wanting the exact minimum with no approximation tolerance is an argument *for* the closed form. Solving the normal equations gives the exact optimum in one shot, with no

Back to 1st Page

learning rate to tune and no stopping criterion to get wrong. Gradient descent only ever approaches the optimum asymptotically.

(D) — false, and also argues the opposite. If X has orthonormal columns then $X^\top X = I$ and the closed form collapses to

$$\hat{\mathbf{w}} = X^\top \mathbf{y}$$

No inversion is needed at all, the d^3 term vanishes, and the problem is perfectly conditioned with $\kappa = 1$. This is the best possible case for the closed form. (It is also the best possible case for gradient descent, since circular contours mean the negative gradient points straight at the minimum — but there is no reason to iterate when a single matrix–vector product gives the answer.)

Note. *A third strong argument for gradient descent is not listed among the options: many losses have no closed form at all. L_1 regression, lasso and logistic regression all lack one, so the machinery of this section is not a convenience there but the only available route. That is why the block matters well beyond linear regression.*

Q7. Fit $\hat{y} = wx$ to the points $(1, 3)$ and $(2, 5)$, starting at $w = 1$ with $\alpha = 0.1$. Using $\frac{\partial J}{\partial w} = -\frac{1}{n}\sum_i \big(y^{(i)} - \hat{y}^{(i)}\big)x^{(i)}$, what is w after one step?
Answer: (B)

Solution. Step 1 — predictions at $w = 1$.

$$\hat{y}^{(1)} = (1)(1) = 1, \qquad \hat{y}^{(2)} = (1)(2) = 2$$

Step 2 — residuals.

$$y^{(1)} - \hat{y}^{(1)} = 3 - 1 = 2, \qquad y^{(2)} - \hat{y}^{(2)} = 5 - 2 = 3$$

Step 3 — gradient. Weight each residual by its own feature value and average over $n = 2$:

$$\frac{\partial J}{\partial w} = -\frac{1}{2}\Big[(2)(1) + (3)(2)\Big] = -\frac{1}{2}(2 + 6) = -\frac{8}{2} = -4$$

Step 4 — update.

$$w \leftarrow 1 - (0.1)(-4) = 1 + 0.4 = 1.4$$

The sign is the whole question. Two minus signs are in play: the formula for $\partial J/\partial w$ already carries one, and the update subtracts the gradient. They cancel, so the correction is *added*. This makes sense — both predictions currently undershoot their targets, so w must increase.

Back to 1st Page

(A) 0.6 — **both signs mishandled**, moving away from the data. **(C)** 0.2 **and (D)** 1.8 correspond to doubling the step in each direction, as from forgetting the $1/n$ average.

Note. *The optimal weight here is* $\hat{w} = \sum x_i y_i / \sum x_i^2 = (3+10)/(1+4) = 13/5 = 2.6$*, so a single step has moved from* 1 *to* 1.4*, covering a quarter of the distance. The multiplier picture confirms this:* J *is quadratic in* w *with curvature* $\frac{1}{n}\sum x_i^2 = 2.5$*, giving an error multiplier of* $1-(0.1)(2.5) = 0.75$ *per step. Indeed* $2.6 - 1.4 = 1.2 = 0.75 \times 1.6$*.*

Q8. Run gradient descent on $J = 5w_1^2 + w_2^2$ from $(1, 1)$ with $\alpha = 0.1$. Where does one step land?

Answer: (A)

Solution. The two coordinates do not interact, since $\partial J/\partial w_1$ involves only w_1 and $\partial J/\partial w_2$ only w_2. Each therefore evolves independently, and for $J = \sum_i c_i w_i^2$ the update is a pure multiplication:

$$w_i \leftarrow w_i - \alpha(2c_i w_i) = (1 - 2\alpha c_i)\, w_i$$

Coordinate w_1, with $c_1 = 5$:

$$1 - 2(0.1)(5) = 1 - 1 = 0 \qquad \Longrightarrow \qquad w_1 = 0 \times 1 = 0$$

Coordinate w_2, with $c_2 = 1$:

$$1 - 2(0.1)(1) = 0.8 \qquad \Longrightarrow \qquad w_2 = 0.8 \times 1 = 0.8$$

So one step lands at $(0,\ 0.8)$. The steep coordinate reaches its optimum *exactly*, in a single step, while the shallow one has barely moved.

(B) $(0.8,\ 0)$ — **the two coordinates swapped.** The larger coefficient produces the steeper wall and hence the larger correction, so it is w_1 that collapses, not w_2. **(C) and (D)** correspond to multipliers that match neither coefficient.

Note. *This single step is the whole problem in miniature. The value* $\alpha = 0.1 = 1/(2c_1)$ *is* perfect *for* w_1 *and poor for* w_2*, which at* $r = 0.8$ *needs roughly* 31 *further steps to reduce its error a thousandfold. Raising* α *to help* w_2 *pushes* w_1*'s multiplier negative, and beyond* $\alpha = 0.2$ *it exceeds* 1 *in magnitude and* w_1 *diverges — one diverging coordinate is enough to destroy the run. The best compromise is* $\alpha = 1/(c_1 + c_2) = 1/6$*, giving both multipliers magnitude* $2/3$*. That gap between what the two coordinates want is measured by* $\kappa = c_1/c_2 = 5$*, and it is exactly what feature scaling exists to reduce.*

Back to 1st Page

Q9. What is a weakness of stopping when the gradient norm falls below a fixed threshold?

Answer: (C)

Solution. The gradient carries the units of the loss divided by those of the weights, so a fixed threshold such as 10^{-6} means different things on different problems. Multiply the entire loss by 1000 — which changes nothing about where the minimum lies — and the same criterion becomes a thousand times stricter. Rescale the features and the gradient components rescale inversely, with the same effect. The criterion is **not scale-free**.

The standard remedy is a *relative* threshold, stopping when

$$\frac{\|\nabla L(\theta_t)\|}{\|\nabla L(\theta_0)\|} < \epsilon$$

which is dimensionless and therefore portable across problems and parametrisations.

(A) — false. The criterion tests against ϵ, not against zero, so it triggers perfectly well; indeed the gradient does become arbitrarily small as the optimum is approached.

(B) — false, and backwards. A small gradient norm is if anything prone to stopping *too early*, not too late: it fires at saddle points and on flat plateaus where the gradient is small but the optimum is distant.

(D) — false. Convexity is irrelevant to whether the criterion can be evaluated. Convexity does affect its *reliability* — on a convex loss a small gradient really does mean proximity to the global minimum — but it is not a requirement for the criterion to work.

Note. *Contrast this with the "small change in loss" criterion, whose weakness is different in kind: it fires on flat plateaus because slow progress is not the same as proximity. Both criteria measure genuine proxies for convergence, and both fail in ways that a fixed iteration budget does not — which is the case for combining them rather than choosing one.*

Q10. With $n = 1000$ and a mini-batch size of 50, how many weight updates occur over 10 epochs?

Answer: (B)

Solution. Step 1 — mini-batches per epoch.

$$\left\lceil \frac{1000}{50} \right\rceil = 20$$

Step 2 — one update per mini-batch, over 10 epochs.

$$20 \times 10 = 200$$

Back to 1st Page

(A) 10 — **the epoch count**, which would be the update total under batch gradient descent. **(C)** 10,000 — **the number of examples processed** (1000×10), the update total under pure SGD. **(D)** 50 — **the batch size**, which is not an update count at all.

The three quantities to keep distinct are the *epoch* (one full pass over the data), the *iteration* or update (one application of the update rule), and the *gradient evaluation* (one per-example derivative). Here there are 10 epochs, 200 iterations and 10,000 gradient evaluations.

Note. *Note that all three variants perform the same* 10,000 *gradient evaluations over* 10 *epochs. What differs is the grouping:* 10 *updates of* 1000 *gradients each,* 200 *updates of* 50 *each, or* 10,000 *updates of* 1 *each. The ceiling function matters when B does not divide n — with $n = 1000$ and $B = 300$ an epoch contains $\lceil 1000/300 \rceil = 4$ updates, the last on a partial batch of* 100*.*

Q11. How does the per-epoch cost of stochastic gradient descent compare with that of batch gradient descent?

Answer: (B)

Solution. Both methods process every example exactly once per epoch, so both compute n per-example gradients at a cost of $O(d)$ each:

$$\text{per-epoch gradient work} = O(nd) \quad \text{for both}$$

The difference is not the amount of work but **what that work buys**. Batch descent spends the entire budget assembling one exact gradient and takes a single well-aimed step. SGD spends the same budget on n separate noisy gradients and takes n rough steps.

SGD usually makes more progress per epoch, but not because it is cheaper — it is because n approximate steps typically cover more ground than one exact step, especially early in training when high precision is wasted anyway.

(A) — false, and the most common misconception. SGD is n times cheaper *per step*, which is a different claim entirely. Per epoch the totals are identical. Confusing the two leads to the belief that SGD gets something for nothing.

(C) — false. Batch descent updates once per epoch but still computes all n gradients to do so. The update itself, an $O(d)$ vector operation, is negligible beside the $O(nd)$ of forming the gradient.

(D) — false. Both scale linearly in d; there is no extra factor.

Back to 1st Page

Note. *SGD's genuine advantages lie elsewhere. It needs only one example in memory at a time, so it handles datasets that do not fit in RAM and data arriving as a stream. It also begins improving the weights after a single example rather than after a full pass, which matters enormously when an epoch takes hours. And its gradient noise acts as a mild regulariser on non-convex problems. None of these is a cost advantage per epoch.*

Q12. You have $n = 1{,}000{,}000$ rows and $d = 20$ features, all in memory. Which method is preferable?

Answer: (A)

Solution. Substitute into the closed-form cost $O(nd^2 + d^3)$:

$$nd^2 = 10^6 \times 400 = 4 \times 10^8, \qquad d^3 = 8000$$

The total is dominated by the nd^2 term at around 4×10^8 operations — a fraction of a second on modern hardware — and the matrix to be inverted is a trivial 20×20. The closed form gives the *exact* optimum in one shot, with no learning rate to tune and no stopping criterion to get wrong.

(B) — false, and the key misconception. The closed-form cost is only *linear* in n, so a million rows is not an obstacle. Cost grows as nd^2, not n^2 or n^3, and $X^\top X$ remains 20×20 no matter how many rows are accumulated into it — it could even be built in a single streaming pass without holding all the data at once.

(C) — false. With $d = 20$, $d^3 = 8000$: entirely negligible. The d^3 term becomes prohibitive when d reaches the tens of thousands, not at 20.

(D) — false. The problem is small by modern standards on both axes.

Note. *The instructive point is which dimension is dangerous. Doubling n roughly doubles the cost; doubling d multiplies the solve by about 8. So the closed form is comfortable with tall, narrow data and helpless with wide data — which is the exact complement of gradient descent's $O(nd)$ per iteration, where n is the burden and d is cheap. One further practical note: even here, one should not form $(X^\top X)^{-1}$ explicitly. Solving the system directly costs the same but is numerically far better behaved, since building $X^\top X$ squares the condition number.*

Q13. Which of the following statements about stopping criteria are correct? (Choose all the correct answers.)

Answer: (A, B, C, D, E)

Solution. (A) — true. A fixed budget is trivially simple and always terminates, but it is blind to what the optimiser is actually doing: it may halt while the loss is still falling steeply, or grind through hundreds of pointless iterations after convergence.

Back to 1st Page

(B) — true. Small change in loss measures *progress*, not proximity. Along a long flat valley — the signature of an ill-conditioned problem — the loss barely moves per step while the weights remain far from the optimum, so the criterion reports a convergence that has not happened. Tightening the threshold delays the problem without fixing it, because the criterion is measuring the wrong quantity.

(C) — true. The gradient carries units, so a fixed threshold is not comparable across problems. Rescaling the loss or the features changes what a given ϵ means without changing the optimisation problem at all.

(D) — true, and the fix for (C). A relative threshold — comparing to the initial gradient norm, or to the current parameter magnitude — is dimensionless and so transfers between problems and parametrisations.

(E) — true. Training loss and generalization part company at some point; monitoring validation loss and stopping when it turns upward halts training at approximately the best generalizing model. Early stopping is itself a form of regularization.

(F) — false. A small gradient norm occurs at *every* stationary point: minima, maxima and saddles alike. In high dimensions saddle points vastly outnumber local minima, so this failure is common in practice. The statement holds for least squares only because convexity guarantees a unique stationary point — but that is a property of that particular loss, and the option claims it "for any loss function".

Note. *Each of the five valid criteria measures a proxy that correlates with convergence without being equivalent to it, and each fails differently. In practice one combines them: an iteration cap as a hard stop, a relative gradient tolerance for genuine convergence, and validation-based early stopping to decide which iterate to keep.*

Q14. With n examples and mini-batch size B, which of the following are correct? (Choose all the correct answers.)

Answer: (A, B, C, D, E)

Solution. (A) — true. This is the definition: one epoch is one complete pass over the training set.

(B) — true. Each mini-batch produces exactly one update, and a pass through n examples in batches of B requires $\lceil n/B \rceil$ of them. The ceiling accounts for a final partial batch when B does not divide n.

(C) and (D) — both true. These are the two endpoints of (B). Batch gradient descent is $B = n$, giving $\lceil n/n \rceil = 1$ update. Pure SGD is $B = 1$, giving n updates.

Back to 1st Page

(E) — true, and the point most often missed. Every variant computes one gradient per example per epoch, so all three perform exactly n gradient evaluations — $O(nd)$ of arithmetic. They differ only in how those evaluations are grouped before the update rule is applied. SGD is not cheaper per epoch; it simply spends the same budget on many rough steps instead of one exact one.

(F) — false. An iteration is one weight update; an epoch is one pass over the data. They coincide only for batch gradient descent, where $B = n$. With mini-batches an epoch contains many iterations, and the distinction matters when reading papers, since "trained for 100k steps" and "trained for 100k epochs" differ by orders of magnitude.

Note. *A useful cross-check on all four true statements: with $n = 1000$ and $B = 50$, one epoch is 20 iterations and 1000 gradient evaluations. Change B to 1 and it becomes 1000 iterations and still 1000 evaluations; change it to 1000 and it becomes 1 iteration and still 1000 evaluations. The evaluation count is invariant, which is exactly statement (E).*

Q15. Which of the following statements about batch, stochastic and mini-batch gradient descent are correct? (Choose all the correct answers.)

Answer: (A, B, C, D, F)

Solution. (A) — true. All three touch every example once per epoch, so all three do $O(nd)$ of gradient work. The difference is the grouping, not the total.

(B) — true. With the full dataset there is no sampling noise, so the computed direction is the exact steepest descent direction. For a step size within the stability range this guarantees the loss decreases at every step, which is why batch descent produces the clean textbook loss curve against which SGD's jaggedness stands out.

(C) — true, and the reason SGD works at all. Sampling an example uniformly at random gives

$$\mathbb{E}[\nabla J_i(\mathbf{w})] = \frac{1}{n}\sum_{i=1}^{n} \nabla J_i(\mathbf{w}) = \nabla J(\mathbf{w})$$

Each step is wrong individually but right on average, so the errors cancel over many steps and the iterates head in the correct direction.

(D) — true. Averaging B estimates cuts the variance by a factor B, and the batch is processed as a single matrix operation, which suits vectorised hardware. Mini-batching improves both stability and throughput simultaneously.

Back to 1st Page

(F) — **true.** With a constant α the gradient noise never vanishes, so the iterates bounce in a region around the optimum whose size scales with α. Decaying the learning rate shrinks that region over time, allowing the iterates to settle.

(E) — **false, and it inverts SGD's chief practical advantage.** SGD requires only one example — or one mini-batch — in memory at a time. This is precisely what makes it viable for datasets larger than RAM and for data arriving as a stream. It is *batch* gradient descent that needs the whole dataset available for every single step.

Note. *Statements (C) and (F) fit together into the standard convergence story for SGD. Unbiasedness means the method is aimed correctly; bounded variance means the noise does not grow; and a schedule satisfying $\sum_k \alpha_k = \infty$ with $\sum_k \alpha_k^2 < \infty$ — large enough steps to travel any distance, small enough in aggregate for the noise to die out — gives convergence to the optimum. The classic choice is $\alpha_k \propto 1/k$.*

Back to 1st Page

6 Feature Scaling

Key ideas for this section

1. **The two transformations.** Min-max maps to a fixed interval, standardization to zero mean and unit variance:

$$x' = \frac{x - \min}{\max - \min} \in [0, 1], \qquad z = \frac{x - \mu}{\sigma}$$

   Both are *affine*, so neither changes the shape of a distribution — skew and outliers survive intact.

2. **Why it helps optimisation.** The entries of $X^\top X$ involve *products* of feature values, so a scale ratio of r between two columns becomes a curvature ratio of about r^2. Scaling collapses that ratio, rounding the contours so a single α suits every coordinate.

3. **Plain OLS is invariant.** Rescaling a feature by c divides its coefficient by c, leaving fitted values, residuals and R^2 untouched. The coefficients are precisely the part that absorbs the rescaling — which is what *produces* the invariance.

4. **Regularization is not invariant.** Ridge and lasso penalise $\|\mathbf{w}\|$ directly, so an arbitrary choice of units decides how hard each coefficient is shrunk. Standardize before regularizing.

5. **Fit the scaler on the training split only.** Statistics computed over training and test combined leak information and bias the reported error downward. Inside cross-validation, the scaler belongs inside each fold.

6. **Two things not to scale.** The intercept column of ones, and any constant column (its σ is zero).

Q1. A feature ranges from 10 to 50 in the training set. What is the min-max scaled value of $x = 30$?

Answer: (B)

Solution. Apply the definition, subtracting the minimum and dividing by the *range*:

$$x' = \frac{x - \min}{\max - \min} = \frac{30 - 10}{50 - 10} = \frac{20}{40} = 0.5$$

The answer is unsurprising once noticed: 30 sits exactly halfway between 10 and 50, and an affine map preserves relative position, so it must land halfway between 0 and 1. This is worth remembering as a check — min-max scaling relocates and rescales the axis but never rearranges the points on it.

Back to 1st Page

(A) 0.6 — **divides by the maximum instead of the range**, computing 30/50 and forgetting to subtract the minimum. **(C)** 0.4 — **subtracts the minimum but still divides by the maximum**, computing $(30-10)/50$. Both errors come from confusing max with max − min; they agree only when min = 0. **(D)** 0.3 corresponds to no consistent calculation on these numbers.

Note. *The denominator is the range, and it is determined by exactly two observations — the smallest and the largest. That is the structural reason min-max scaling is fragile: a single extreme value dictates the scale for the entire column, as the fourth question in this section shows. Standardization is steadier because σ averages over every observation, so one wild value moves it far less.*

Q2. A training feature has mean 70 and standard deviation 10. What is the z-score of $x = 85$?

Answer: (C)

Solution. Subtract the mean, then divide by the standard deviation:

$$z = \frac{x-\mu}{\sigma} = \frac{85-70}{10} = \frac{15}{10} = 1.5$$

The value sits one and a half standard deviations above the mean. Note what the z-score buys: it is *dimensionless*. Whatever units x was measured in cancel between numerator and denominator, which is exactly why standardized coefficients become comparable across features while raw ones do not.

Applied to an entire column, the transformation gives that column mean 0 and standard deviation 1:

$$\mathbb{E}[z] = \frac{\mathbb{E}[x]-\mu}{\sigma} = 0, \qquad \mathrm{Var}[z] = \frac{\mathrm{Var}[x]}{\sigma^2} = 1$$

(A) 0.15 — **divides by σ twice**, i.e. 15/100. **(B)** 8.5 — **forgets to subtract the mean**, computing 85/10. **(D)** 15 — **forgets to divide by σ**, leaving the raw deviation from the mean, which still carries the original units.

Note. *Option (D) is the most instructive slip:* 15 *is a perfectly meaningful quantity — the deviation in the feature's own units — but it is not standardized, because the division by σ is what removes the units. Centring alone gives mean zero; it is the scaling step that makes different features commensurable.*

Back to 1st Page

Q3. What is the key difference between min-max scaling and standardization?

Answer: (B)

Solution. Min-max output is confined to $[0, 1]$ by construction: the training minimum maps to 0, the training maximum to 1, and everything between lands in between. Standardization offers no such guarantee — a value five standard deviations from the mean maps to 5, and there is no upper limit at all. **Bounded versus unbounded** is the essential difference.

(A) — false, and the most seductive misconception. Neither method makes a distribution normal. Both are *affine* maps of the form $x \mapsto ax + b$, which relocate and rescale the axis without rearranging any points on it. A strongly skewed column is exactly as skewed after scaling as before; what changes is the numbers on the axis, not the shape of the histogram. Making a distribution more symmetric requires a genuinely non-linear transformation such as a logarithm or a Box–Cox power transform.

(C) — false, and reversed. Both transformations are affine. Neither is non-linear, and the option assigns the property to the wrong one besides.

(D) — false. Neither method removes anything. An outlier remains an outlier under both; the two differ only in how much damage it does to the rest of the column, which is the subject of the next question.

Note. *The bounded range in (B) is why min-max is preferred where a fixed input domain is required — image pixel intensities, or activations feeding a bounded non-linearity. Standardization is preferred almost everywhere else, both because it is less sensitive to extremes and because unit variance is the natural scale for the curvature argument that motivates scaling in the first place.*

Q4. A feature has values mostly between 1 and 10, plus one recording error of 1000. What happens under min-max scaling?

Answer: (C)

Solution. The range is now dictated entirely by the erroneous value:

$$\max - \min = 1000 - 1 = 999$$

So the genuine data, spanning 1 to 10, maps to

$$\frac{1-1}{999} = 0 \qquad \text{through} \qquad \frac{10-1}{999} \approx 0.009$$

Every real observation is compressed into roughly the first 1% of the interval, where the differences between them become vanishingly small relative

Back to 1st Page

to the scale. The outlier alone occupies the remaining 99%. **The resolution among the values that matter has been destroyed** by a single bad point.

(A) — **false.** Min-max is the *least* robust common scaler, precisely because its denominator depends on only two observations. Standardization is steadier here — one extreme value among many raises σ but does not hijack it the way it hijacks a range — though it is not immune either. Genuinely robust alternatives use the median and interquartile range in place of the mean and standard deviation.

(B) — **false, and inverted.** Under min-max the maximum maps to 1, not 0. The outlier is the maximum, so it lands at 1 while everything else clusters near 0 — the opposite of the spread the option describes.

(D) — **false.** The range is well defined and non-zero, so the transformation completes without error. That is the difficulty: it fails silently, producing a numerically valid column that has lost the information it was supposed to carry.

Note. *The practical lesson is ordering. Outlier detection belongs* before *scaling, not after, since scaling on contaminated data bakes the contamination into the transformation itself — and the stored* min *and* max *then get applied to every future observation. A related edge case: a genuinely constant column has* max = min*, giving division by zero. Such columns carry no information and are normally dropped.*

Q5. One feature has values near 1 and another near 1000. What is the rough condition number of $X^\top X$?

Answer: (B)

Solution. The entries of $X^\top X$ are inner products of feature columns:

$$(X^\top X)_{jk} = \sum_{i=1}^{n} x_j^{(i)} x_k^{(i)}$$

Every entry is therefore a sum of *products* of feature values. The diagonal entry for the small feature is of order $n \cdot 1^2 = n$, while that for the large feature is of order $n \cdot 1000^2 = 10^6 n$. The curvature ratio is thus

$$\kappa \approx \frac{10^6 n}{n} = 10^6$$

The scale disparity is squared. A ratio of 1000 between the features becomes a ratio of 10^6 between the curvatures.

This is catastrophic for gradient descent. The contours are ellipses stretched by a factor of $\sqrt{10^6} = 1000$ along one axis, and the stability bound

Back to 1st Page

$\alpha < 1/\lambda_{\max}$ is set by the steep direction while progress in the shallow direction is governed by $\lambda_{\min}$. Any α safe for the first leaves the second essentially frozen.

(A) 1000 — **the feature ratio itself, not squared.** This is the most common answer and it misses the squaring, which is exactly the point of the question. **(C)** — **false**, and states the opposite of the truth: unequal scaling is the primary source of ill-conditioning. **(D)** 2000 — **adds the two scales**, but the condition number is a ratio of eigenvalues, not a sum of anything.

Note. *This is why the unscaled example in the lecture notes detonates within five iterations rather than merely converging slowly. It also explains a numerical point that has nothing to do with gradient descent: forming $X^\top X$ squares the condition number of X, so even a direct solve loses precision on badly-scaled data. That is the argument for QR or SVD factorisations of X over explicitly building the Gram matrix.*

Q6. Why does feature scaling speed up gradient descent?

Answer: (C)

Solution. Scaling equalises the diagonal entries of $X^\top X$, which pulls the elongated elliptical contours toward circles. Two things improve at once. First, the condition number $\kappa = \lambda_{\max}/\lambda_{\min}$ falls, so a **single α can serve every coordinate** instead of being capped by the steepest while the shallowest crawls. Second, on rounder contours the negative gradient **points closer to the minimum**, rather than across a narrow valley, so the trajectory stops zigzagging.

(A) — **false, and the key conceptual trap.** Scaling does *not* move the minimum. Under plain OLS the fitted values, residuals and R^2 are identical before and after scaling; only the coefficients change, absorbing the rescaling so that the products $w_j x_j$ stay fixed. Scaling improves *how you get there*, never *where you end up*.

(B) — **false.** The squared-error loss is $\|\mathbf{y} - X\mathbf{w}\|^2$, a convex quadratic with curvature matrix $X^\top X$, which is positive semi-definite whatever the columns contain. Convexity holds regardless of scaling and is not something scaling supplies.

(D) — **false.** The per-iteration cost is $O(nd)$ either way, since the same matrix–vector products are computed on the same-sized arrays. What falls is the *number* of iterations required, not the cost of each.

Note. *The distinction in (D) is worth stating precisely: scaling changes the iteration count, not the cost per iteration. And the distinction in (A) is the one this whole section rests on — for unregularized OLS, scaling is*

Back to 1st Page

purely an optimisation convenience. The moment a penalty is added, it also changes the answer.

Q7. A feature with standard deviation σ is standardized. Under plain OLS, what happens to its coefficient?

Answer: (A)

Solution. Write the standardized feature as $x' = (x - \mu)/\sigma$, so that $x = \sigma x' + \mu$. Since OLS reaches the same fitted values either way, the contribution of this feature must be unchanged. Substituting,

$$wx = w(\sigma x' + \mu) = \underbrace{(w\sigma)}_{w'} x' + \underbrace{w\mu}_{\text{absorbed by the intercept}}$$

so the new coefficient is

$$w' = w\sigma$$

It is multiplied by σ. A feature with $\sigma = 20$ has its coefficient multiplied by 20.

The direction is worth pinning down by intuition rather than memory. Standardizing a widely-spread feature *shrinks* its numerical values, so its coefficient must *grow* to compensate and keep the product fixed. Dividing the feature by σ multiplies the weight by σ.

The interpretation shifts accordingly: w' is the change in $\hat{y}$ per *one-standard-deviation* rise in the feature. That is what makes standardized coefficients comparable to one another — they are all quoted per standard deviation rather than per unit of whatever the feature happens to be measured in.

(B) — the right factor, wrong direction, and the answer to the reverse question of what happens when a *feature* is scaled by c. **(C) — false**: OLS is invariant in its predictions, not in its coefficients. **(D) — false**: quadratic scaling applies to variances, $\mathrm{Var}[aX] = a^2 \mathrm{Var}[X]$, and to entries of $X^\top X$; a coefficient is a rate of change and scales linearly.

Note. *To recover an original-unit coefficient from a standardized fit, invert the relation: $w = w'/\sigma$. This matters when reporting results, since a coefficient of "12,000 per standard deviation of floor area" is not a statement a client can act on. Note also that this is the same invariance as the earlier question on scaling a feature by c — standardization is just the case $c = 1/\sigma$ combined with a shift.*

Q8. After standardizing all features, what does the fitted intercept equal?

Answer: (B)

Back to 1st Page

Solution. Standardizing centres every feature column at zero. Now use a property of any least-squares fit with an intercept: because the residuals are orthogonal to the ones column, they sum to zero, and consequently the fitted surface passes through the point of means. Evaluating the model there,

$$\bar{y} = w_0 + \sum_j w_j \, \bar{x}'_j$$

and after centring every $\bar{x}'_j = 0$, so every term in the sum vanishes:

$$w_0 = \bar{y}$$

The intercept becomes the mean of the target.

This gives standardized coefficients a clean reading: the prediction for an observation at the average of every feature is simply the average target, and each w'_j says how far $\hat{y}$ moves from that average when feature j shifts by one standard deviation with all others held at their means.

(A) — false, and confuses the features with the target. Centring is applied to X, not to $\mathbf{y}$. The intercept would be zero only if the target were centred as well — which is in fact common practice, and then the intercept can be dropped from the model entirely. **(C) — false**: the intercept absorbs the shift, changing from w_0 to $w_0 + \sum_j w_j \mu_j$. **(D) — false**: the feature means enter the calculation, but the intercept is the target mean, not a sum of feature means.

Note. *Two practical corollaries. First, the ones column must not itself be scaled — standardizing a constant column would divide by $\sigma = 0$, and centring it would annihilate the intercept altogether. Second, this is one of the two reasons the intercept is excluded from regularization penalties: it now represents the target's overall level, which there is no reason to shrink toward zero.*

Q9. Plain OLS is invariant to feature rescaling, yet ridge regression is not. Why?

Answer: (C)

Solution. The invariance argument for OLS works because the objective depends on $\mathbf{w}$ only through the product $X\mathbf{w}$. Replacing X with XM for invertible M gives the solution $M^{-1}\mathbf{w}$, and $(XM)(M^{-1}\mathbf{w}) = X\mathbf{w}$, so the fit is untouched.

Ridge adds a term that breaks this:

$$J(\mathbf{w}) = \|\mathbf{y} - X\mathbf{w}\|^2 + \lambda \|\mathbf{w}\|_2^2$$

The penalty is a function of $\mathbf{w}$ *directly*, not of $X\mathbf{w}$, so the substitution no longer leaves it invariant. Concretely: a feature measured in metres

Back to 1st Page

receives a coefficient 100 times larger than the same feature in centimetres, and since the penalty is quadratic it therefore absorbs 10^4 times more of the penalty budget. **An arbitrary choice of units ends up deciding which features get shrunk.** Standardizing first puts every coefficient on comparable footing so that λ applies evenly.

(A) — **false.** $X^\top X + \lambda I$ is invertible for every $\lambda > 0$ regardless of scaling, since $X^\top X$ is positive semi-definite and adding λI shifts all its eigenvalues up by λ. Guaranteed invertibility is a virtue of ridge, not a consequence of scaling.

(B) — **false.** Ridge has the closed form $\hat{\mathbf{w}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$ on any data whatsoever, scaled or not.

(D) — **false.** Ridge imposes no constraint on the feature range. Standardization is advisable, but the $[0, 1]$ interval specifically is not required by anything in the method.

Note. *This is the sharpest statement of when scaling is optional and when it is not. For plain OLS it changes the route but not the destination, so skipping it costs only iterations. For ridge and lasso it changes the destination — a different model is fitted — so skipping it is a modelling error. The same sensitivity is inherited by anything that behaves like an implicit regularizer, including early stopping: run to convergence, gradient descent reaches the invariant OLS minimum, but stopped at a fixed budget, where you land depends on the geometry that scaling alters.*

Q10. You compute μ and σ over the combined training and test data before splitting. What is wrong with this?

Answer: (B)

Solution. The test set exists to estimate performance on data the model has never seen. Computing μ and σ over all rows means those statistics encode information about the test observations, and that information is then baked into every training feature the model fits on. **The reported test error is optimistically biased** — it no longer estimates what it is meant to estimate.

The correct procedure is to fit the scaler on the training split alone and then apply those stored statistics, unchanged, to the validation and test data:

$$\mu, \sigma \text{ from train} \quad \longrightarrow \quad z_{\text{test}} = \frac{x_{\text{test}} - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

This mirrors deployment, where future observations must be transformed using statistics fixed at training time, because the future data is not available when the scaler is fitted.

Back to 1st Page

(A) — false, and the standard rationalisation. Determinism is not the issue, and neither is the absence of labels. μ and σ are quantities *derived from data you are pretending not to have seen*, and leakage is about information flow, not about whether a computation involves the target.

(C) — a true observation, but not the objection. It is correct that the training features will no longer have exactly zero mean, since they were centred using a mean computed over a larger set. But this is a harmless numerical side effect of no practical consequence. Mistaking it for the problem is to notice the symptom and miss the disease.

(D) — false. The coefficients remain perfectly interpretable, just expressed relative to statistics that were computed improperly. Interpretability is not what leakage damages; the honesty of the error estimate is.

Note. *The version of this error most often committed in practice is subtler than the one described: scaling the whole training set once and then running k-fold cross-validation on it. Each fold's held-out portion contributed to the statistics used to transform its own training portion, so the CV estimate is quietly inflated. The scaler belongs* inside *the fold, like every other fitted step — which is precisely what pipeline abstractions in libraries such as scikit-learn exist to enforce.*

Q11. You fit a min-max scaler on the training set. A test observation exceeds the training maximum. What happens?

Answer: (C)

Solution. The scaler stores the training minimum and maximum and applies them unchanged. With a training range of $[10, 50]$, a test value of 60 maps to

$$\frac{60-10}{50-10} = \frac{50}{40} = 1.25$$

The value lands above 1, correctly signalling extrapolation beyond anything seen during training.

This is not a bug. The $[0, 1]$ guarantee was always a statement about the *training* data, and a value outside that interval carries genuine information: the model is being asked to predict in a region it was never fitted on. Silently clipping to 1.0 would discard that signal and, worse, would map every out-of-range value to the same point — so a test value of 60 and one of 6000 would become indistinguishable.

(A) — false as a description of the default, though clipping is available as an option in some implementations. It should be a deliberate choice made when the bounded range is a hard requirement downstream,

Back to 1st Page

not an automatic behaviour, and it comes at the cost of the information just described.

(B) — false, and it is leakage. Refitting on test data is exactly the error of the previous question. It would also mean the transformation applied to the training data no longer matches the one applied at test time, so the model would be evaluated on inputs measured on a different scale from the ones it learned.

(D) — false. The arithmetic is well defined for any input; the transformation simply returns a value outside $[0, 1]$.

Note. *This behaviour makes min-max scaling a free extrapolation detector: monitoring how often scaled test values fall outside* $[0, 1]$*, and by how far, is a cheap check for distribution shift in production. Standardization gives the same signal in a different currency — a* z*-score of* 8 *on live data is a warning that the incoming distribution has drifted from the one the model was trained on.*

VisionGATE
UNLEASH YOUR POTENTIAL, ACHIEVE YOUR VISION

Back to 1st Page